

SCORECARD V4 — RE-GRADED AFTER FIXES · SAME BENCHMARK, NO CURVE

Observe + traceAI Docs Scorecard v4

Final hygiene pass — full frontmatter on all 41 auto leaves, TODO markers cleared, Tags over its word floor. Scores hold at v3 (those are merge-gate items the 1-5 cards don't measure).

4.40_{/5}

vs v3

OVERALL · 35 PAGES

4.50_{/5}

vs v3

OBSERVE · 24 PAGES

4.18_{/5}

vs v3

TRACEAI · 11 PAGES

0 pages moved up · 0 down · 14 now at 5/5

Still under the 4–5 bar (0) — with the honest reason

Graded 2026-06-19 (v2) · v3 baseline: 3.60 / 3.79 / 3.18 · 0 pages 404 · method: §4 8-dimension rubric via Layer A cards + Section C (QB) minimums, rendered pages screenshotted at localhost

Observe — product section

24 pages · v3 average 4.50/5 (v3 4.50, ▲ +0.00)

Future AGI

DocsIntegrationsCookbooksSDKAPI

Search...#KAsk AI

Request demoDashboard >

Observe

OBserve

Overview

GETTING STARTED

Quick start

CONCEPTS

Observability model

Traces

Spans

Sessions and users

OpenTelemetry

traceAI SDK

FEATURES

Trace explorer

Sessions

Users

Tags

Dashboards

Alerts and monitors

Trace evals

Voice observability

Manual tracing

REFERENCE

Filter syntax

Dashboard metrics

Export and endpoints

TROUBLESHOOTING

No traces appear

Spans and attributes

Dashboard numbers

Alerts not firing

Docs / Observe: monitor LLM apps in production

Copy page

Observe: monitor LLM apps in production

FutureAGI Observe is production monitoring for LLM apps — replay any request, track quality, cost, and latency, score responses, and get alerted on issues.

About

Observe is where you monitor your AI application in production. When an answer is wrong, a request is slow, or cost spikes, it shows you exactly what your app did on that request — so you find the cause instead of guessing, and catch problems before your users report them.

You can replay any request end to end, watch quality, cost, and latency trends, group activity by conversation or by end user, score responses automatically with evals, and get alerted the moment something degrades.

Under the hood, Observe records each request as a trace — but day to day you work with the views built on top: the trace explorer, sessions, users, evals, dashboards, and alerts. If you're starting fresh, send your first trace; if traces are already flowing, open the trace explorer.

Future AGI

BackError Feed Demo

Auto refresh (10s)

Full 300FilterDisplayAdd Eval

Get started

Primary Graph

Latency

Latency and Tokens

Build

Dataset

Prompts

Agents

Industries

Knowledge base

Entity

Key

Observe

Tracing

Error Feed

Tasks

Users

Annotations

Alerts

Dashboard

Gateway

Gateway

Resources

Docs

Help

Trace Name

Input

Output

Timestamp

Status

Latency

Tokens

support_agent.run

"Is my dealer close?"

"Your order will arrive in 10 minutes."

Apr 27, 2025 12:31:28

OK

5.5s

support_agent.run

"The food is cold and inedible."

"I see your order had an issue. I'll connect you with a customer support agent."

Apr 27, 2025 12:15:47

OK

7.4s

support_agent.run

"My order is at the lobby bar."

"I'll connect you with a customer support agent."

Apr 27, 2025 11:23:22

OK

10.2s

support_agent.run

"Can you tell me what's wrong with my order?"

"Your order will arrive in 10 minutes."

Apr 27, 2025 08:51:01

OK

6.4s

support_agent.run

"The food is cold and inedible."

"I'm having trouble processing your request. I'll connect you with a customer support agent."

Apr 27, 2025 09:22:01

WARN

18.8s

support_agent.run

"When will my order arrive?"

"I see your order had an issue. I'll connect you with a customer support agent."

Apr 27, 2025 04:16:51

OK

5.7s

support_agent.run

"It's been 45 minutes and my order hasn't arrived yet."

"I'll connect you with a customer support agent."

Apr 27, 2025 06:59:41

OK

9.6s

support_agent.run

"Can't add a tip after delivery."

"I see your order had an issue. I'll connect you with a customer support agent."

Apr 27, 2025 05:34:53

OK

8.8s

support_agent.run

"The food is cold and inedible."

"I'll connect you with a customer support agent."

Apr 27, 2025 05:24:06

OK

10.1s

support_agent.run

"It's been 45 minutes and my order hasn't arrived yet."

"Per our policy, refunds are processed within 30 days of delivery. I'll connect you with a customer support agent."

Apr 27, 2025 04:51:09

OK

7.9s

support_agent.run

"It's been 45 minutes and my order hasn't arrived yet."

"I'll connect you with a customer support agent."

Apr 27, 2025 03:48:17

OK

10.0s

support_agent.run

"Driver marked delivered but I haven't received the package yet."

"I'll connect you with a customer support agent."

Apr 27, 2025 02:33:25

OK

11.2s

Every production request, captured as a trace and ready to inspect, group, score, or alert on.

words: 829 w (floor 400–900) · code lines: 0 · screenshots: 1 · tables: 1 · H2: 8 · Mermaid: 0 · TOC:

yes

SCORE RATIONALE + GAP (V2)

829w with a dedicated 'why this matters in production' section, card grid, action-first routing, right-rail TOC — meets the overview bar.

Future AGI

Docs

Integrations

Cookbooks

SDK

API

Search...

#K

Ask AI

Request demo

Dashboard >

Observe

OBserve

Overview

GETTING STARTED

Quick start

CONCEPTS

Observability model

Traces

Spans

Sessions and users

OpenTelemetry

traceAI SDK

FEATURES

Trace explorer

Sessions

Users

Tags

Dashboards

Alerts and monitors

Trace evals

Voice observability

Manual tracing

REFERENCE

Filter syntax

Dashboard metrics

Export and endpoints

TROUBLESHOOTING

No traces appear

Spans and attributes

Dashboard numbers

Alerts not firing

Docs / Observe / Getting started with Observe

Copy page

Getting started with Observe

Start monitoring your AI app in production with Observe — send your first trace in five minutes, learn the core concepts, and jump straight to the integration guide for your framework.

Getting started

Observe shows you what your AI application actually did in production — every request, replayable, with quality, cost, and latency attached. The fastest way in is to send one trace and watch it land. This page is the on-ramp: the five-minute quick start, the concepts the rest of Observe is built on, and the integration guide for whatever framework you already run.

Once traces are flowing, the same data drives everything else in Observe: you debug a single bad answer in the [trace explorer](#), watch cost and latency trends on [dashboards](#), score quality with [trace evals](#), and get paged when a metric slips with [alerts](#). You instrument once, and every view reads the same underlying traces.

There are three ways to start, depending on where you are:

New to Observe?

Run the [quick start](#) — install, register a project, send one request, and confirm it lands, in about five minutes.

Already on a framework?

Skip the manual setup and add its instrumentor below — one line and your existing calls are traced.

Want the mental model first?

Read the [observability model](#) to see how traces, spans, sessions, and evals relate before you wire anything up.

Start here

Quick start: send your first trace

Install traceAI, register a project, run one request, and confirm the trace in about five minutes.

ON THIS PAGE

Getting started

Start here

Set up your framework

Learn the concepts

Where next

Edit on GitHub

Send feedback

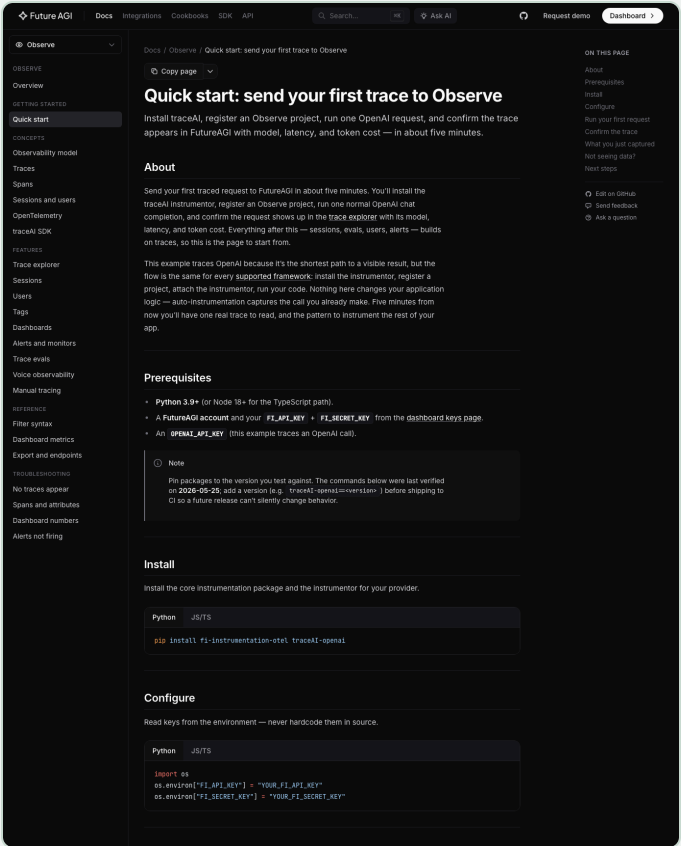
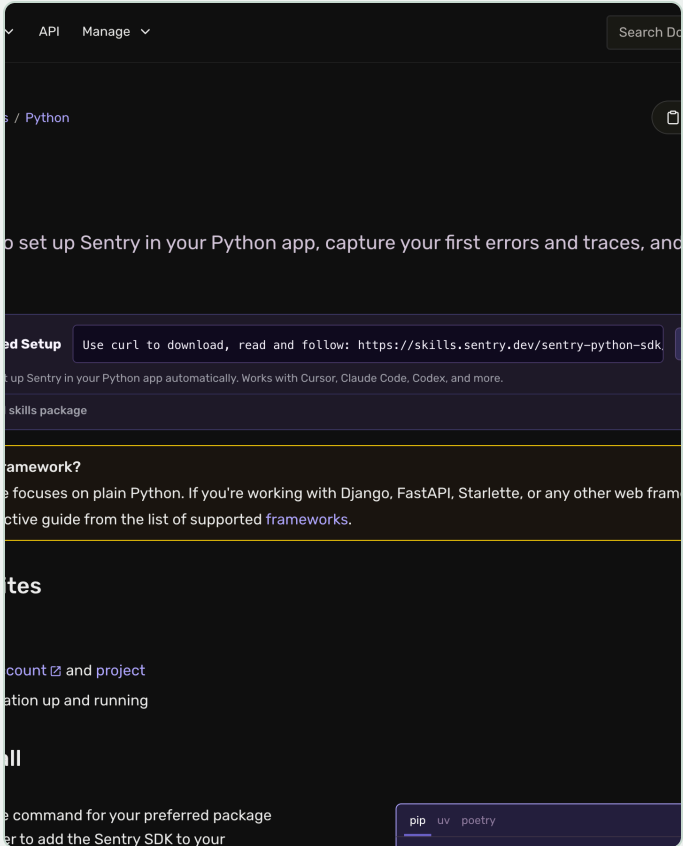
Ask a question

words: 448 w (floor 400–900) · code lines: 0 · screenshots: 0 · tables: 0 · H2: 5 · Mermaid: 0 · TOC:

yes

SCORE RATIONALE + GAP (V2)

448w (clears the 400w floor) with a three-paths framing + concept/integration card grids; could add one result screenshot to reach 5.

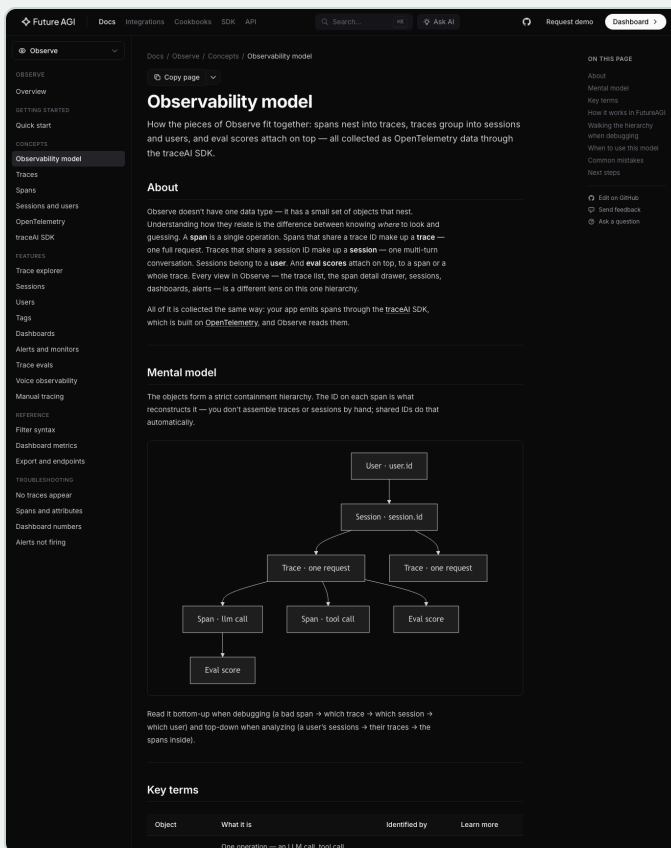


words: 803 w (floor 800–1200) · code lines: 46 · screenshots: 1 · tables: 1 · H2: 9 · Mermaid: 0 · TOC: yes

SCORE RATIONALE + GAP (V2)

803w (clears 800 floor), numbered steps with expected-output, Verify, 'What you just captured'; single reused trace-list screenshot (not annotated) keeps it off 5.

OUR PAGE (V4) – OBSERVABILITY MODEL

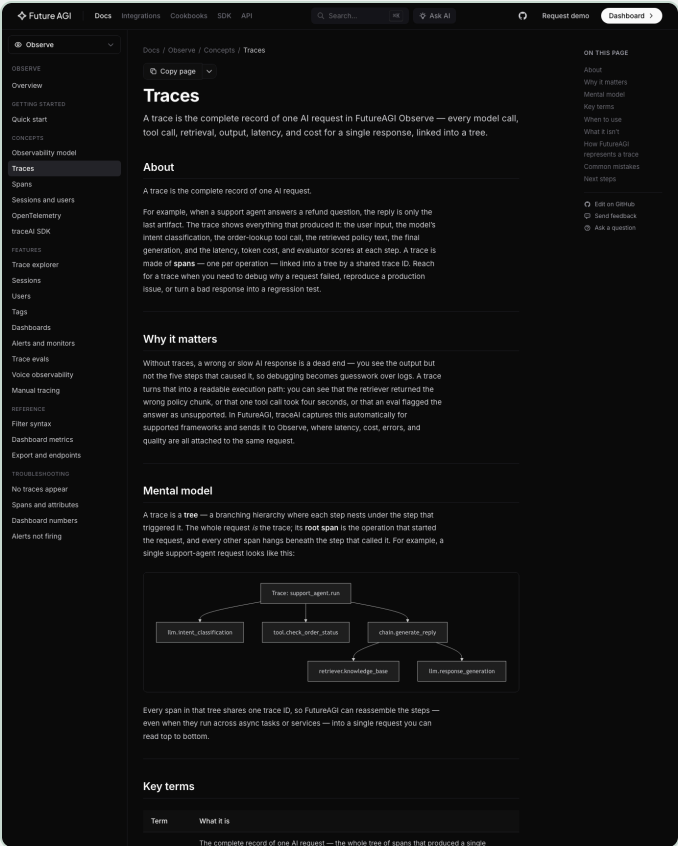
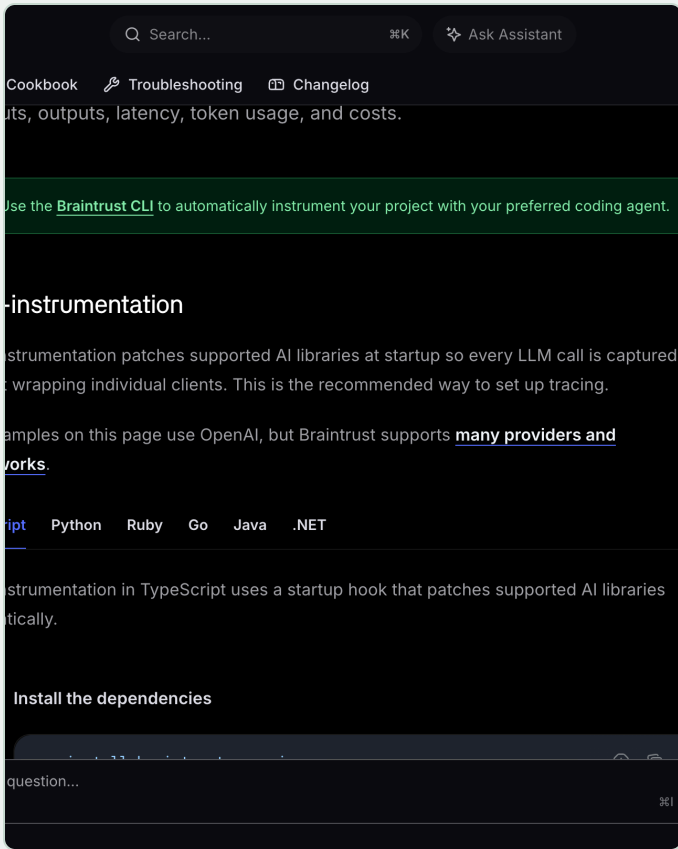


921w, two Mermaid diagrams, key-terms table, when-to-use + common-mistakes, code ≤ 30 — full concept shape.

CARD 3 · CONCEPT TIER 1 FLAGSHIP · PAIR 4 — BRAINTRUST DENSITY

BENCHMARK EXEMPLAR · PAIR 4 — BRAINTRUST DENSITY
· 5/5

OUR PAGE (V4) — TRACES



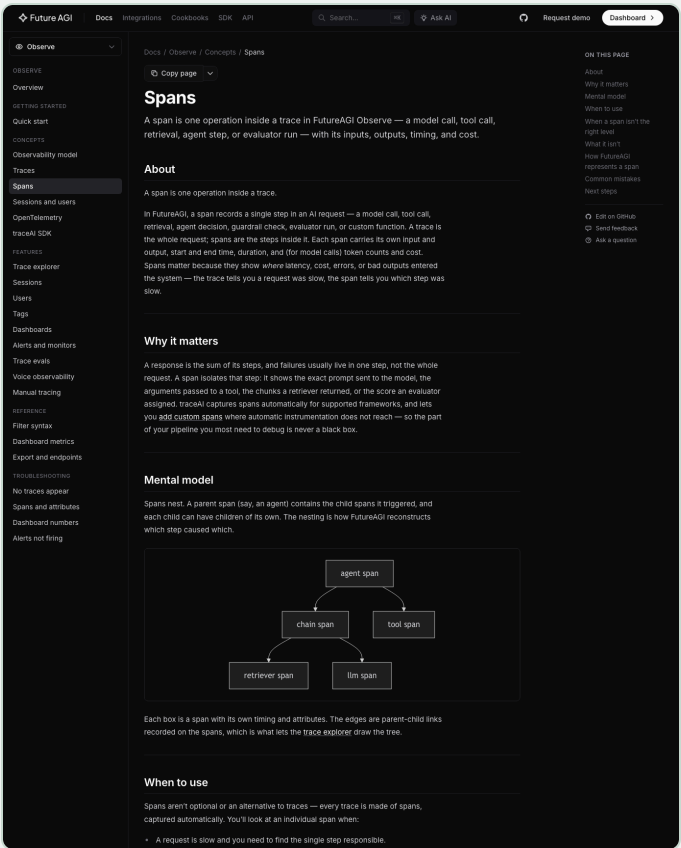
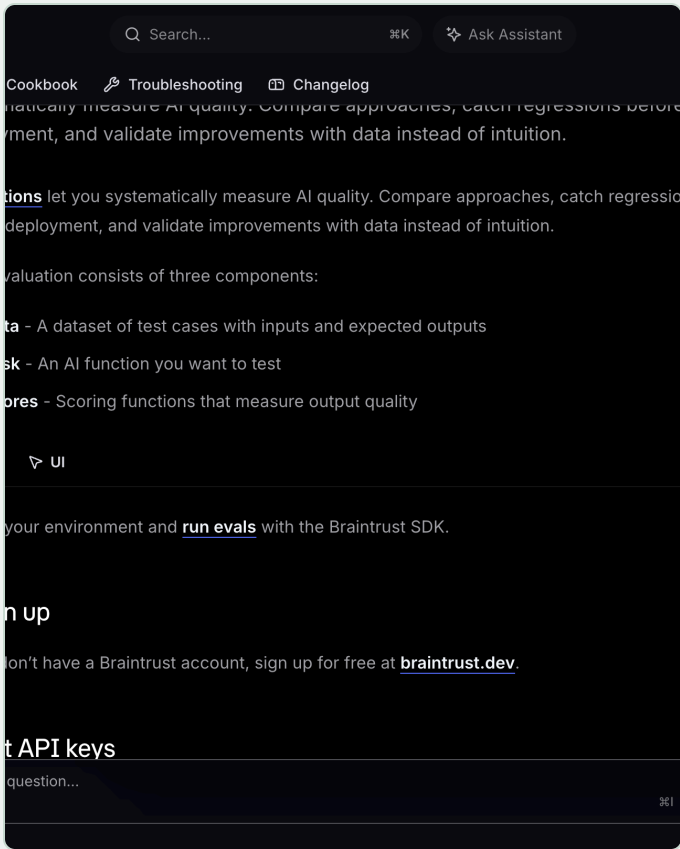
words: 931 w (floor 800–1500) · code lines: 8 · screenshots: 0 · tables: 2 · H2: 9 · Mermaid: 1 · TOC: yes

SCORE RATIONALE + GAP (V2)

931w, definition-first, Mermaid, when-to/when-not, related cards — meets the concept bar.

BENCHMARK EXEMPLAR · PAIR 4 — BRAINTRUST DENSITY · 5/5

OUR PAGE (V4) — SPANS



words: 975 w (floor 800–1500) · code lines: 13 · screenshots: 0 · tables: 1 · H2: 9 · Mermaid: 1 · TOC: yes

SCORE RATIONALE + GAP (V2)

975w, Mermaid, code ≤ 30 , and now an explicit 'when a span isn't the right level' section.

Future AGI

Docs

Integrations

Cookbooks

SDK

API

Search...

#K

Ask AI

Request demo

Dashboard >

Observe

OVERVIEW

GETTING STARTED

CONCEPTS

FEATURES

REFERENCE

TROUBLESHOOTING

Observe

Overview

Quick start

Observability model

Traces

Spans

Sessions and users

OpenTelemetry

traceAI SDK

Trace explorer

Sessions

Users

Tags

Dashboards

Alerts and monitors

Trace evals

Voice observability

Manual tracing

Filter syntax

Dashboard metrics

Export and endpoints

No traces appear

Spans and attributes

Dashboard numbers

Alerts not firing

Docs / Observe / Concepts / Sessions and users

Copy page

Sessions and users

Sessions and users are how Observe groups traces — a session is one multi-turn conversation, a user is one end user across all their sessions and traces.

About

Sessions and users are the two ways Observe groups individual traces into something larger.

A single **trace** is one request. But a **chatbot** conversation is many requests, and one customer has many conversations. A **session** ties together the traces of one multi-turn interaction — a whole support chat, start to finish. A **user** ties together every session and trace belonging to one end user. You create both by setting an attribute on your spans (`session.id` and `user.id`); Observe does the grouping. Use them when a single trace is too small a unit — when you need a customer's whole history, not one message.

Why it matters

Most real problems span more than one request. A bug report says “the assistant kept losing track” — that is a *session* problem, visible only when you read the conversation in order. A cost spike traces back to *one user* hammering an expensive model — visible only when you roll traces up per user. Without these groupings you are stuck reading isolated requests and reconstructing context by hand. With them, Observe shows per-session and per-user metrics — trace count, latency, cost, eval pass-rate — so behavior over time becomes legible.

Mental model

Users contain sessions; sessions contain traces; traces contain spans. It is one hierarchy, built from two attributes you set.

User (user.id)

ON THIS PAGE

About

Why it matters

Mental model

When to use

What it isn't

How FutureAGI represents sessions and users

How the IDs propagate

When not to use sessions and users

Common mistakes

Next steps

Edit on GitHub

Send feedback

Ask a question

words: 1086 w (floor 800–1500) · code lines: 13 · screenshots: 0 · tables: 1 · H2: 10 · Mermaid: 1 · TOC: yes

SCORE RATIONALE + GAP (V2)

1086w, Mermaid, ID-propagation explanation, explicit 'when not to use' — full concept shape.

Future AGI

Docs

Integrations

Cookbooks

SDK

API

Search...

#K

Ask AI

Request demo

Dashboard >

Observe

OBSERVE

Overview

GETTING STARTED

Quick start

CONCEPTS

Observability model

Traces

Spans

Sessions and users

OpenTelemetry

traceAI SDK

FEATURES

Trace explorer

Sessions

Users

Tags

Dashboards

Alerts and monitors

Trace evals

Voice observability

Manual tracing

REFERENCE

Filter syntax

Dashboard metrics

Export and endpoints

TROUBLESHOOTING

No traces appear

Spans and attributes

Dashboard numbers

Alerts not firing

Docs / Observe / Concepts / OpenTelemetry

Copy page

OpenTelemetry

OpenTelemetry is the open, vendor-neutral standard Observe uses to collect and export traces — your app emits OTel spans that the traceAI SDK sends to Observe.

About

OpenTelemetry (OTel) is the open, vendor-neutral standard for collecting traces, metrics, and logs from software. It defines how a span is structured and how spans are exported, so any tool that speaks OTel can read them. FutureAGI is built on it: your app emits OTel spans, and [traceAI](#) exports them over OTLP to Observe. Because it's a standard, the same instrumentation works across languages, frameworks, and backends.

Why it matters

Building on a standard is what keeps you un-locked-in. Your instrumentation isn't proprietary to FutureAGI — the same OTel spans can go to any OTel-compatible backend, and FutureAGI can ingest spans from anything that emits them. OTel is also built for scale: batch export handles high-volume tracing without blocking your app. So the choice to standardize on OTel is why "set up tracing once" works regardless of your stack.

Mental model

OTel sits between your app and FutureAGI: your code (or an instrumentor) creates spans, a processor batches them, and an exporter ships them over OTLP.

Your app

OTel SDK: create spans

Span processor: batch

OTLP HTTP/gRPC

FutureAGI backend

Observe

traceAI plugs into this pipeline — it's the layer that creates *LLM-shaped* spans and configures the exporter to point at FutureAGI.

What OTel gives traceAI

ON THIS PAGE

About

Why it matters

Mental model

What OTel gives traceAI

Key terms

When to use

When not to use raw OpenTelemetry directly

What it isn't

How FutureAGI uses OpenTelemetry

Common mistakes

Next steps

Edit on GitHub

Send feedback

Ask a question

words: 986 w (floor 800–1500) · code lines: 15 · screenshots: 0 · tables: 1 · H2: 11 · Mermaid: 1 · TOC:

yes

SCORE RATIONALE + GAP (V2)

986w (clears 800 floor) with 'what OTel gives traceAI' + explicit when-not; single diagram and Tier-2 depth keep it at 4.

Future AGI

Docs

Integrations

Cookbooks

SDK

API

Search...

#K

Ask AI

Request demo

Dashboard >

Observe

OBSERVE

Overview

GETTING STARTED

Quick start

CONCEPTS

Observability model

Traces

Spans

Sessions and users

OpenTelemetry

traceAI SDK

FEATURES

Trace explorer

Sessions

Users

Tags

Dashboards

Alerts and monitors

Trace evals

Voice observability

Manual tracing

REFERENCE

Filter syntax

Dashboard metrics

Export and endpoints

TROUBLESHOOTING

No traces appear

Spans and attributes

Dashboard numbers

Alerts not firing

Docs / Observe / Concepts / traceAI SDK

Copy page

traceAI SDK

traceAI is FutureAGI's open-source instrumentation SDK on OpenTelemetry — it captures LLM, tool, and retrieval calls as standardized spans that Observe reads as traces.

About

traceAI is the instrumentation SDK; Observe is the product that reads its traces. traceAI is FutureAGI's open-source instrumentation SDK, built on OpenTelemetry. It's a set of conventions and per-framework instrumentors that capture what your AI app does — model calls, tool calls, retrievals, agent steps — and map them to standardized span attributes. Add the instrumentor for your framework, and those calls become traces in Observe without hand-writing spans. traceAI is natively supported by FutureAGI but emits standard OTel, so it works with any OTel-compatible backend too.

Why it matters

Raw OpenTelemetry knows nothing about LLMs — it has no concept of a prompt, a completion, token cost, or a tool call. traceAI fills that gap: it turns framework calls into LLM-shaped spans with consistent keys, so a LangChain trace and an OpenAI trace look the same in Observe and are queryable the same way. That standardization is what makes filtering, evals, and dashboards work across different stacks.

Mental model

traceAI is the adapter between your framework and OpenTelemetry: the instrumentor wraps the framework, produces standardized spans, and hands them to the OTel pipeline that exports to FutureAGI.

Framework call: OpenAI, LangChain, ...

traceAI instrumentor

Standardized OTel spans

FutureAGI Observe

You pick the instrumentor that matches your framework; the rest of the pipeline is the same OTel flow for everyone.

ON THIS PAGE

About

Why it matters

Mental model

SDK vs product, auto vs manual

When to use

When not to use traceAI

What it isn't

How FutureAGI represents traceAI

Common mistakes

Next steps

Edit on GitHub

Send feedback

Ask a question

words: 932 w (floor 800–1500) · code lines: 13 · screenshots: 0 · tables: 1 · H2: 10 · Mermaid: 1 · TOC:

yes

SCORE RATIONALE + GAP (V2)

932w (clears floor), SDK-vs-product + register() explanation + when-not; Tier-2 concept, solid 4.

CARD 4 · FEATURE DEEP-DIVE

TIER 1

Future AGI

Docs

Integrations

Cookbooks

SDK

API

Search...

#K

Ask AI

Request demo

Dashboard >

Observe

Observe

Overview

GETTING STARTED

Quick start

CONCEPTS

Observability model

Traces

Spans

Sessions and users

OpenTelemetry

traceAI SDK

FEATURES

Trace explorer

Sessions

Users

Tags

Dashboards

Alerts and monitors

Trace evals

Voice observability

Manual tracing

REFERENCE

Filter syntax

Dashboard metrics

Export and endpoints

TROUBLESHOOTING

No traces appear

Spans and attributes

Dashboard numbers

Alerts not firing

Docs / Observe / Features / Trace explorer

Copy page

Trace explorer

Inspect one production AI request in Observe — read its span tree, check latency, cost, tokens and errors, and open any step's exact input and output.

About

The trace explorer is the page in Observe where you debug a single production AI request. When one user hit an error, one answer came back wrong, or one request ran slow, this is where you go to see exactly what your app did.

The unit you work with is a *trace row*. Each row is one complete request — the user input, every model and tool call made along the way, the final output, the latency and token cost of each step, and whether anything failed.

The explorer lists those rows newest-first. Click any row to open its span tree and read the request step by step.

Future AGI

Back

Error Field Demo

Last updated on 2024/02/10/25, 10:13 AM

Auto refresh (10s)

Default Workshop

Get started

Primary Graph

Latency Breakdown

Paul 300

Filter

Display

Add Evals

Falcon AI

Build

Dataset

Prompts

Agents

Prototype

Knowledge base

Events

Keys

Observe

Tracing

Error field

Tasks

Users

Annotations

Alerts

Dashboard

Gateway

Gateway

Resources

Docs

Help

Trace Name

Input

Output

Timestamp

Status

Latency

Tokens

support_agent.run

"Is my cleaver close?"

"Your order will arrive in s...

Apr 27, 2026 12:51:28

OK

5.8s

support_agent.run

"The food is cold and ined...

"I see your order had an is...

Apr 27, 2026 12:16:47

OK

7.4s

support_agent.run

"My order is at the lobby b...

"I'll connect you with a cu...

Apr 27, 2026 11:23:22

OK

10.2s

support_agent.run

"Can you tell me what to do...

"Your order will arrive in s...

Apr 27, 2026 09:51:01

OK

6.4s

support_agent.run

"The food is cold and ined...

"I'm having trouble proces...

Apr 27, 2026 09:22:01

ERROR

16.8s

support_agent.run

"When will my order arrive?"

"I see your order had an is...

Apr 27, 2026 06:19:51

OK

5.7s

support_agent.run

"It's been 45 minutes and ...

"I'll connect you with a cu...

Apr 27, 2026 04:59:41

OK

9.6s

support_agent.run

"Can't add a tip after deliv...

"I see your order had an is...

Apr 27, 2026 00:34:53

OK

8.8s

support_agent.run

"The food is cold and ined...

"I'll connect you with a cu...

Apr 27, 2026 00:34:06

OK

10.1s

support_agent.run

"It's been 45 minutes and ...

"Per our policy, refunds to...

Apr 27, 2026 04:51:09

OK

7.9s

support_agent.run

"It's been 45 minutes and ...

"I'll connect you with a cu...

Apr 27, 2026 03:48:17

OK

10.0s

support_agent.run

"Driver marked delivered ...

"I'll connect you with a cu...

Apr 27, 2026 02:33:25

OK

11.2s

ON THIS PAGE

About

When to use

How it works

Trace list columns

Inspect a trace

Filter and search

Limits and edge cases

Next steps

Edit on GitHub

Send feedback

Ask a question

The trace list, newest request first. Each row is one full request; the Status and Latency columns are the fastest way to spot what broke or ran slow.

When to use

Reach for the trace explorer when you need to look at individual requests:

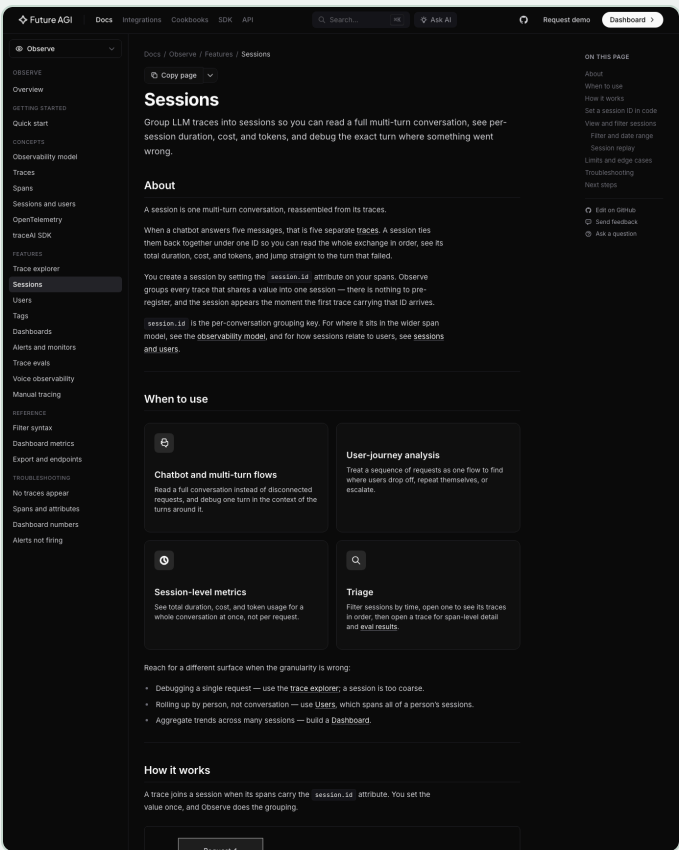
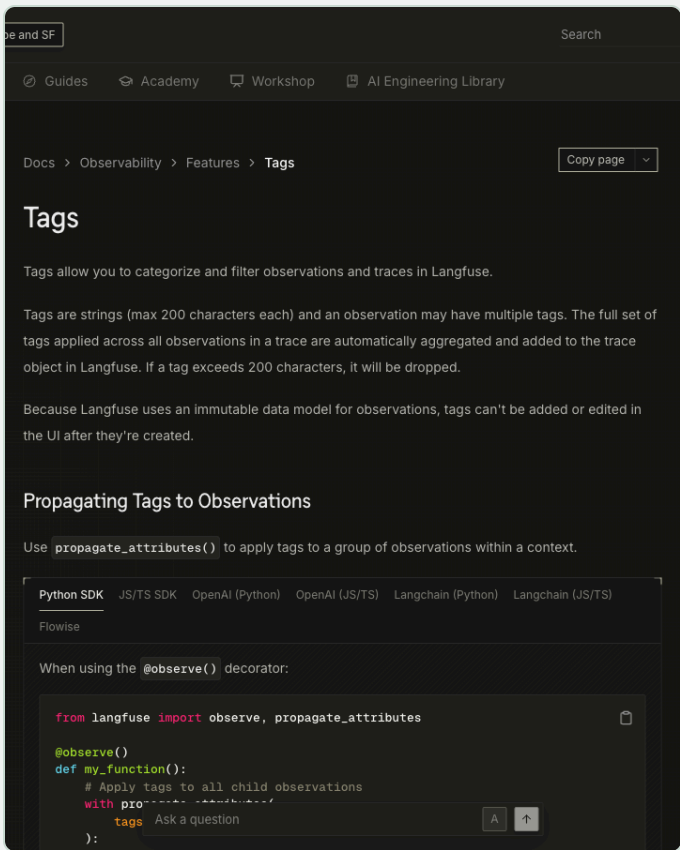
words: 1286 w (floor 900–1800) · code lines: 7 · screenshots: 3 · tables: 2 · H2: 8 · Mermaid: 1 · TOC: yes

SCORE RATIONALE + GAP (V2)

1286w, 3 screenshots (in 2–4), Mermaid, columns + limits tables, full cross-links — the feature exemplar.

BENCHMARK EXEMPLAR · PAIR 1 — LANGFUSE TAGS · 5/5

OUR PAGE (V4) — SESSIONS



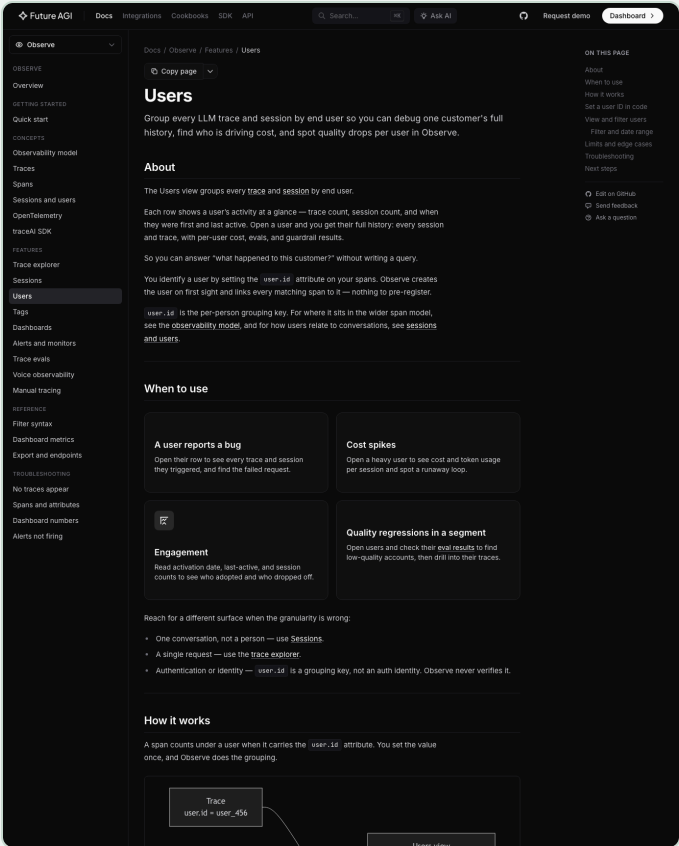
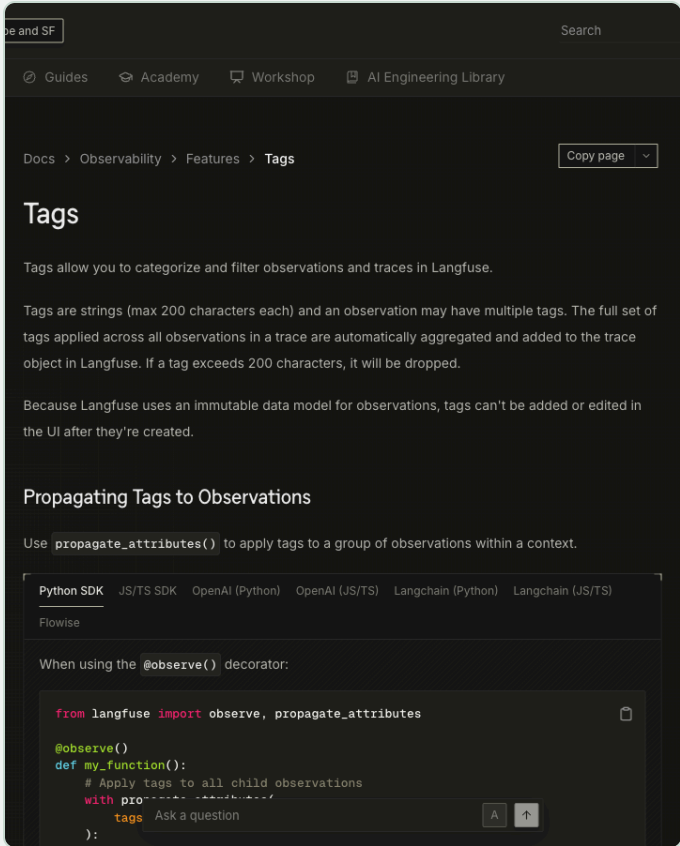
words: 1140 w (floor 900–1800) · code lines: 31 · screenshots: 4 · tables: 2 · H2: 8 · Mermaid: 1 · TOC: yes

SCORE RATIONALE + GAP (V2)

1140w; screenshots trimmed 6→4 (now within the 2–4 cap); tables + code + links present.

BENCHMARK EXEMPLAR · PAIR 1 — LANGFUSE TAGS · 5/5

OUR PAGE (V4) — USERS



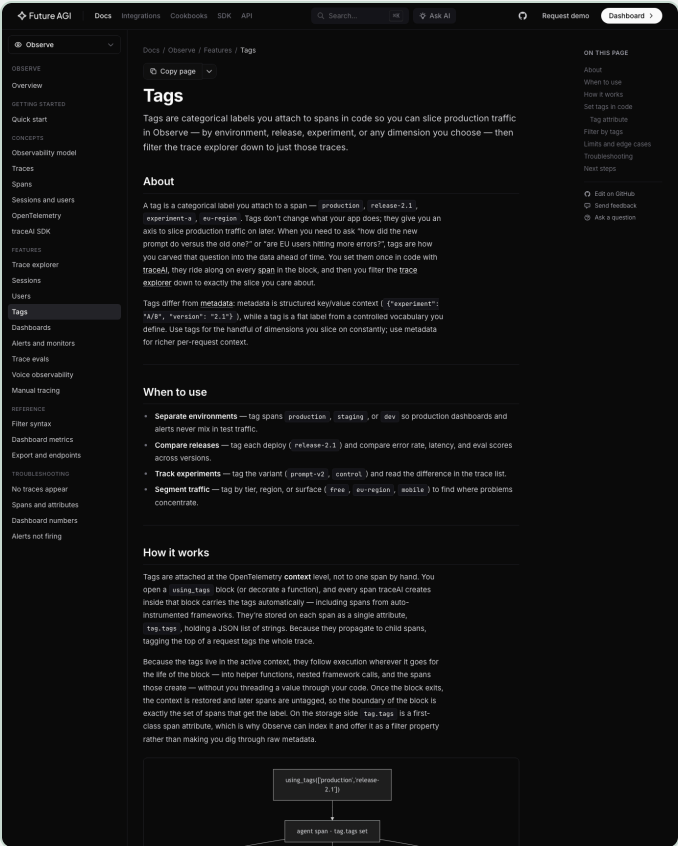
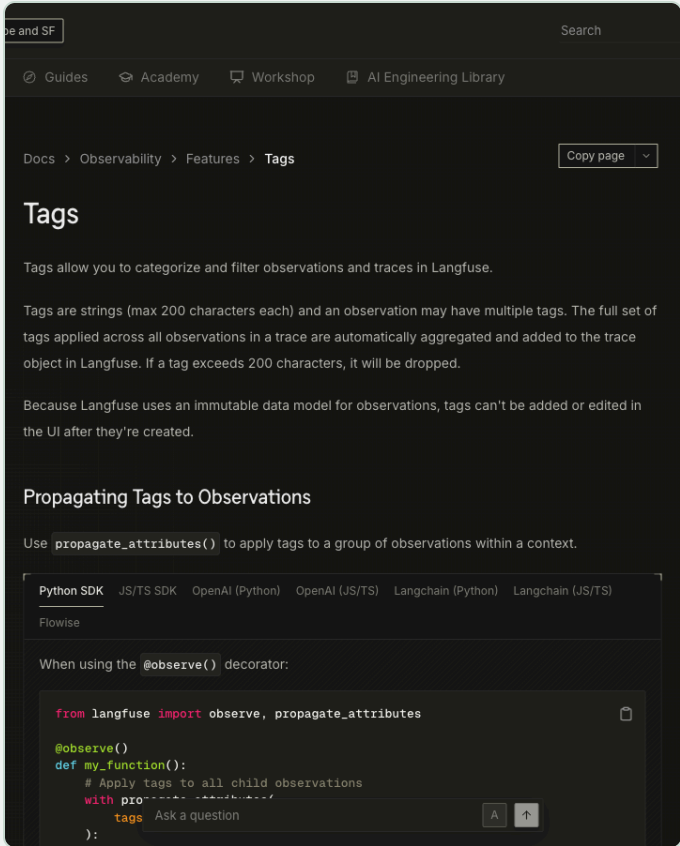
words: 1156 w (floor 900–1800) · code lines: 31 · screenshots: 4 · tables: 2 · H2: 8 · Mermaid: 1 · TOC: yes

SCORE RATIONALE + GAP (V2)

1156w; screenshots trimmed 5→4 (within cap); per-user tables + code present.

BENCHMARK EXEMPLAR · PAIR 1 — LANGFUSE TAGS · 5/5

OUR PAGE (V4) — TAGS



words: 929 w (floor 900–1800) · code lines: 29 · screenshots: 1 · tables: 2 · H2: 8 · Mermaid: 1 · TOC: yes

SCORE RATIONALE + GAP (V2)

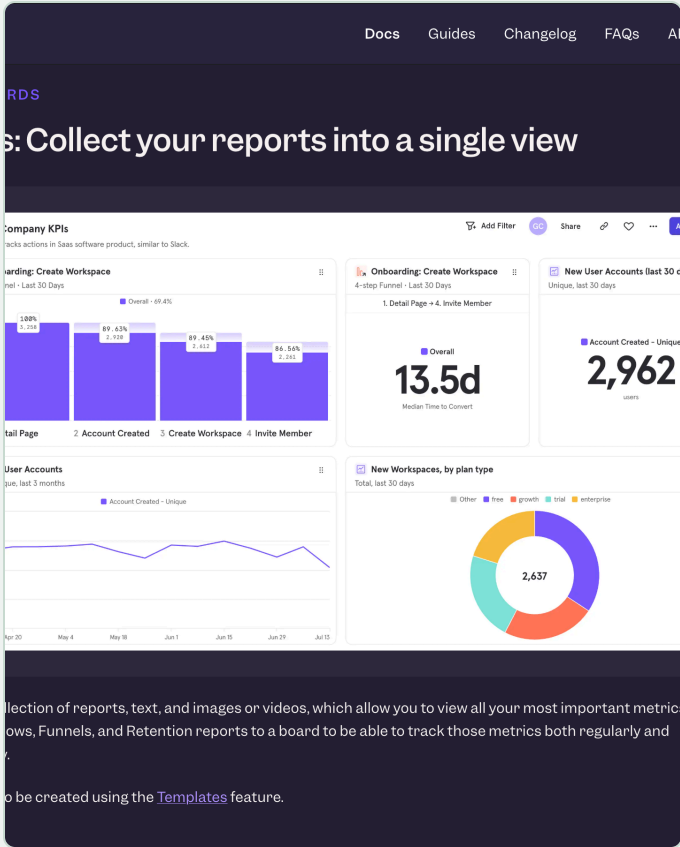
Now 929w (clears the 900-word floor) with propagation diagram, tag-attribute table, 3-tab code, and a troubleshooting table. Held at 4 by screenshot count (1 vs the 2-4 a 5 wants) — the tag-filter-with-results shot is blocked: the shared local stack's postgres is corrupted from an unclean shutdown, and recovering it would disturb another task's data.

CARD 4 · FEATURE DEEP-DIVE

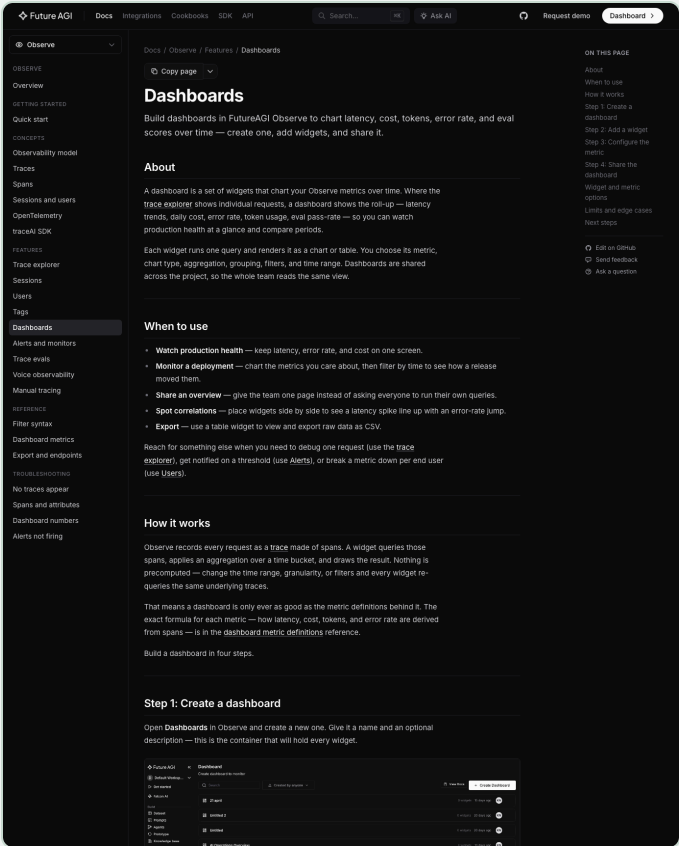
TIER 1

FLAGSHIP · PAIR 2 — MIXPANEL BOARDS

BENCHMARK EXEMPLAR · PAIR 2 — MIXPANEL BOARDS · 5/5



OUR PAGE (V4) — DASHBOARDS



words: 1030 w (floor 900–1800) · code lines: 0 · screenshots: 4 · tables: 2 · H2: 10 · Mermaid: 0 · TOC:

yes

SCORE RATIONALE + GAP (V2)

4-step numbered flow, 4 screenshots, widget-options + limits tables — gold feature page.

Future AGI

Docs

Integrations

Cookbooks

SDK

API

Search...

#K

Ask AI

Request demo

Dashboard >

Observe

OBSERVE

Overview

GETTING STARTED

Quick start

CONCEPTS

Observability model

Traces

Spans

Sessions and users

OpenTelemetry

traceAI SDK

FEATURES

Trace explorer

Sessions

Users

Tags

Dashboards

Alerts and monitors

Trace evals

Voice observability

Manual tracing

REFERENCE

Filter syntax

Dashboard metrics

Export and endpoints

TROUBLESHOOTING

No traces appear

Spans and attributes

Dashboard numbers

Alerts not firing

Docs / Observe / Features / Alerts and monitors

Copy page

Alerts and monitors

Define monitors on Observe metrics — error rate, latency, cost, or eval scores — and get notified by email or Slack when a value crosses a threshold.

About

A monitor watches one Observe metric on a schedule and notifies you when it crosses a threshold. Pick a metric — error rate, latency, cost, or an eval score — set a threshold and direction, and choose where alerts go: email, Slack, or both. When the threshold is breached the monitor creates an alert log and sends the notification. You can review past alerts, mark them resolved, or mute a monitor without deleting it. Monitors are how Observe tells *you* something broke instead of you watching a dashboard.

When to use

- Catch errors early — alert when error rate or LLM API failure rate spikes after a deploy.
- Hold latency limits — alert when response time goes above your target.
- Control cost — warn before token usage hits your budget.
- Guard quality — alert when a pass/fail eval (e.g. toxicity) starts failing more often.

When not to use

- Exploring trends — a monitor is a tripwire, not a chart; use [Dashboards](#).
- Debugging one request — use the [trace explorer](#).
- A non-observe project — monitors run on `observe` projects only; an experiment project has nothing for a monitor to watch.

How it works

A monitor pairs one metric with one threshold rule and runs on a fixed cadence. Each run reads the metric over its window and compares the value to the threshold; when the comparison is true the monitor writes a `UserAlertMonitorLog` record and sends the notification to its configured channels.

ON THIS PAGE

About

When to use

When not to use

How it works

Set up a monitor

Inputs and parameters

Edge cases and limits

Troubleshooting

Related features

Edit on GitHub

Send feedback

Ask a question

words: 1034 w (floor 900–1800) · code lines: 0 · screenshots: 2 · tables: 2 · H2: 9 · Mermaid: 0 · TOC: yes

SCORE RATIONALE + GAP (V2)

1034w (was 773); added how-it-works, modes/options, when-not, two more troubleshooting rows; 2 screenshots.

Future AGI

Docs

Integrations

Cookbooks

SDK

API

Search...

#K

Ask AI

Request demo

Dashboard >

Observe

Observe

Overview

GETTING STARTED

Quick start

CONCEPTS

Observability model

Traces

Spans

Sessions and users

OpenTelemetry

traceAI SDK

FEATURES

Trace explorer

Sessions

Users

Tags

Dashboards

Alerts and monitors

Trace evals

Voice observability

Manual tracing

REFERENCE

Filter syntax

Dashboard metrics

Export and endpoints

TROUBLESHOOTING

No traces appear

Spans and attributes

Dashboard numbers

Alerts not firing

Docs / Observe / Features / Trace evals

Copy page

Trace evals

Score traced spans for hallucination, tone, bias, and more — filter which spans, run historically or continuously, sample to control cost, and read results per span.

About

Evals run automated quality checks on your production traces, scoring each response for hallucination, tone, bias, toxicity, and more. You pick which checks to run, filter which spans they apply to, and choose whether to score existing traffic (historical) or new spans as they arrive (continuous). Scores attach to the span in the [trace explorer](#) and can feed alerts when quality drops.

When to use

- Score production quality — after a release, run a historical eval for hallucinations, bias, or unsafe content across real traffic.
- Catch regressions automatically — a continuous task scores new spans so you see quality drops before users report them.
- Investigate an incident — filter by date range or session to evaluate only the spans from one window or flow.
- Run several checks at once — attach multiple evals so each span is scored for tone, safety, and accuracy in one run.

When not to use

- A one-off check while developing — run an eval inline at the span; see [in-line evaluations](#).
- Watching a score trend over time — chart it on a [Dashboard](#).
- Defining a new evaluator — that lives in the evaluation section; this page only runs evals against traces.

What you can evaluate: span, trace, and session

An eval can run at three levels, depending on what you want to judge:

- Span level — scores a single step on its own: one LLM call, one tool call, one retrieval. Use it to check

ON THIS PAGE

About

When to use

When not to use

What you can evaluate: span, trace, and session

Adding context to an eval

How to run evals

Outputs and interpretation

Edge cases and limits

Related features

Edit on GitHub

Send feedback

Ask a question

words: 1083 w (floor 900–1800) · code lines: 0 · screenshots: 3 · tables: 3 · H2: 8 · Mermaid: 0 · TOC: yes

SCORE RATIONALE + GAP (V2)

1083w, 3 screenshots, span/trace/session-level tables; no code block (UI-driven feature) keeps it at 4.

CARD 4 · FEATURE DEEP-DIVE

TIER 2

Future AGI

Docs

Integrations

Cookbooks

SDK

API

Search...

Ask AI

Request demo

Dashboard

Observe

Observe

Overview

GETTING STARTED

Quick start

CONCEPTS

Observability model

Traces

Spans

Sessions and users

OpenTelemetry

traceAI SDK

FEATURES

Trace explorer

Sessions

Users

Tags

Dashboards

Alerts and monitors

Trace evals

Voice observability

Manual tracing

REFERENCE

Filter syntax

Dashboard metrics

Export and endpoints

TROUBLESHOOTING

No traces appear

Spans and attributes

Dashboard numbers

Alerts not firing

Docs / Observe / Features / Voice observability

Copy page

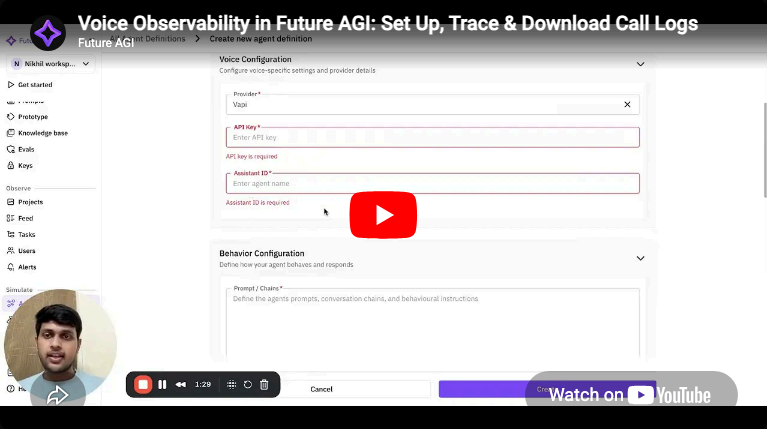
Voice observability

Connect a voice provider like Vapi or Retell and get every call as a trace in Observe — transcript, recording, cost, and duration — with no SDK or code changes.

About

Voice agents are hard to debug: conversations happen in real time, across turns, and you usually hear about failures from a user, not a log. Voice observability pulls call logs from your voice provider into Observe automatically — **no SDK or code changes**. Connect a provider (Vapi, Retell, or Eleven Labs) with its API key and assistant ID, and every call shows up as a **trace** with its transcript, recording URLs, cost, and duration. From there you can run **evals**, set **alerts**, and filter and export, exactly like any other trace.

Voice Observability in Future AGI: Set Up, Trace & Download Call Logs



When to use

- See every voice call for an agent in one project, without instrumenting code.
- Evaluate conversations — run quality, bias, or adherence evals on voice call spans.
- Alert on voice metrics — get notified when a voice project degrades.
- Debug with transcripts and recordings — open a call to read the transcript and play the recording.

ON THIS PAGE

About

When to use

When not to use

Connect a voice provider

How it works

What a voice project shows

Voice call columns

Edge cases and limits

Supported providers

Troubleshooting

Related features

Edit on GitHub

Send feedback

Ask a question

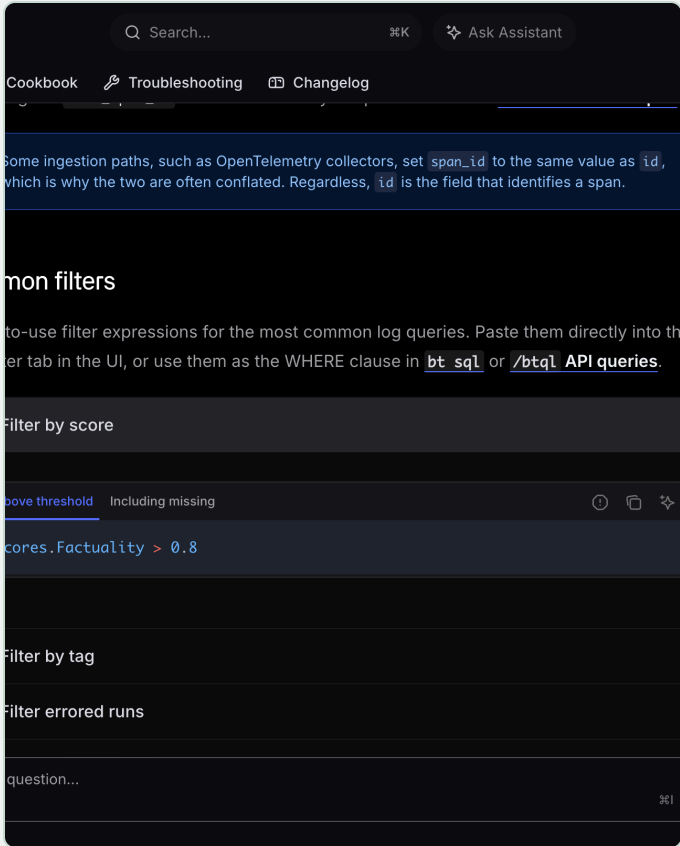
words: 1025 w (floor 900–1800) · code lines: 0 · screenshots: 4 · tables: 3 · H2: 11 · Mermaid: 0 · TOC:

yes

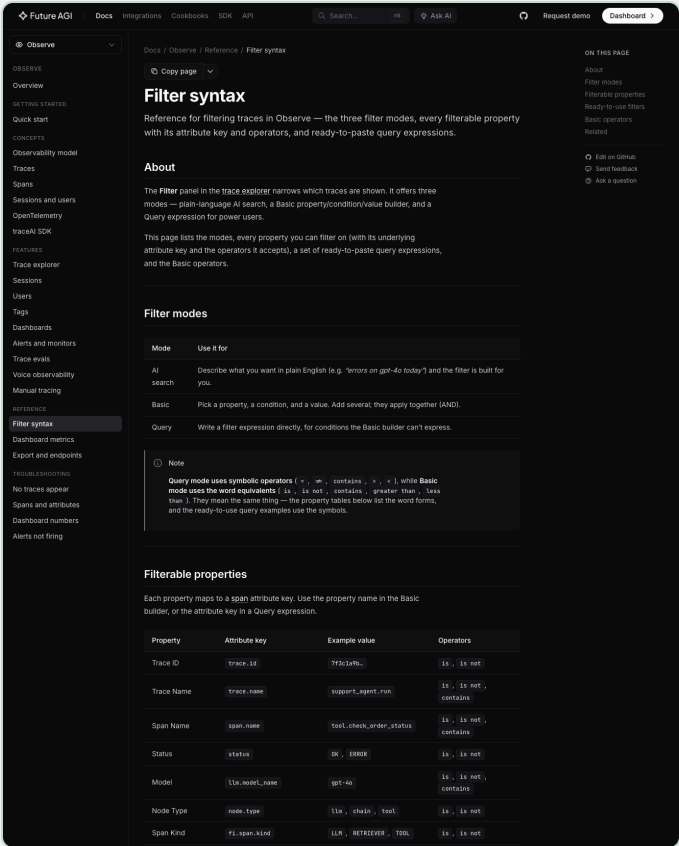
SCORE RATIONALE + GAP (V2)

1025w (was 760), 4 screenshots, per-turn metric tables + what-a-voice-project-shows.

BENCHMARK EXEMPLAR · PAIR 6 — FILTER DEPTH · 5/5



OUR PAGE (V4) — FILTER SYNTAX



words: 694 w · code lines: 6 · screenshots: 0 · tables: 3 · H2: 6 · Mermaid: 0 · TOC: yes

SCORE RATIONALE + GAP (V2)

Real reference — operator/field tables + paste-ready expressions, 0 screenshots (correct for reference).

Future AGI

Docs

Integrations

Cookbooks

SDK

API

Q Search... #K

Ask AI

Request demo

Dashboard >

Observe

OVERVIEW

GETTING STARTED

CONCEPTS

FEATURES

REFERENCE

TROUBLESHOOTING

Overview

Quick start

Observability model

Traces

Spans

Sessions and users

OpenTelemetry

traceAI SDK

Trace explorer

Sessions

Users

Tags

Dashboards

Alerts and monitors

Trace evals

Voice observability

Manual tracing

Filter syntax

Dashboard metrics

Export and endpoints

No traces appear

Spans and attributes

Dashboard numbers

Alerts not firing

Docs / Observe / Reference / Dashboard metrics

Copy page

Dashboard metrics

Reference for the metrics, aggregations, and granularities available in Observe dashboard widgets — what each metric measures and how it's rolled up.

About

Each `dashboard` widget plots one metric, rolled up by an aggregation over a time granularity. This page defines the metrics and the aggregation and granularity options.

Metrics

Metric	Unit	What it measures
Span count	count	Number of spans matching the widget's filters.
Error count	count	Number of spans/traces with <code>ERROR</code> status.
Span response time	ms	Latency of spans.
LLM response time	ms	Latency of LLM spans specifically.
Token usage	tokens	Tokens consumed (prompt + completion).
Cost	USD	Computed cost of model calls.
Eval pass-rate	%	Share of evaluated spans that passed their eval.

The widget editor lists the metrics available for your project.

Note

Eval pass-rate depends on eval sampling — it covers only the spans an eval actually ran on, not all traffic. If the number looks off, see [Dashboard numbers look wrong](#).

How each metric is derived

Every metric is computed from the `spans` that match the widget's filters and time bucket — nothing is precomputed. The raw value per bucket is derived as follows; the `aggregation` you pick (next section) is then applied over that bucket.

ON THIS PAGE

About

Metrics

How each metric is derived

Aggregations

Granularity

Reading a metric correctly

Related

Edit on GitHub

Send feedback

Ask a question

words: 567 w · code lines: 0 · screenshots: 0 · tables: 3 · H2: 7 · Mermaid: 0 · TOC: yes

SCORE RATIONALE + GAP (V2)

567w (was 354) now with a per-metric derivation/formula table + two real consequences; could add a worked example per metric to reach 5.

Future AGI

Docs

Integrations

Cookbooks

SDK

API

Search...

#K

Ask AI

Request demo

Dashboard >

Observe

Observe

Overview

GETTING STARTED

Quick start

CONCEPTS

Observability model

Traces

Spans

Sessions and users

OpenTelemetry

traceAI SDK

FEATURES

Trace explorer

Sessions

Users

Tags

Dashboards

Alerts and monitors

Trace evals

Voice observability

Manual tracing

REFERENCE

Filter syntax

Dashboard metrics

Export and endpoints

TROUBLESHOOTING

No traces appear

Spans and attributes

Dashboard numbers

Alerts not firing

Docs / Observe / Reference / Export and endpoints

Copy page

Export and endpoints

Reference for getting trace data out of Observe — exporting the current view, the available formats, and the OTLP endpoints traces are sent to (cloud and self-hosted).

About

There are two directions for trace data: **out of** Observe (exporting what you're viewing) and **into** Observe (the OTLP endpoints traceAI sends to). This page covers both.

Export from the trace explorer

The download icon in the **trace explorer** header exports the **current view** — the traces that match your active filters and time range.

CSV is the only export format. There is no JSON or Parquet download from the trace explorer.

Format	Use for
CSV	Spreadsheet analysis, sharing, importing elsewhere.

The CSV holds one row per span in the current view, with the columns the explorer shows: trace ID, span name, status, model, provider, latency, token counts, cost, timestamp, and any eval scores present on the span.

Note

The export reflects the current view, so a very large view may be truncated. Narrow the filters or time range to export a complete slice.

Ingestion endpoints

traceAI exports spans over OTLP to FutureAGI. The transport and target are environment-driven:

Variable	Transport	Default
FI_BASE_URL	HTTP collector	FutureAGI cloud collector

words: 579 w · code lines: 0 · screenshots: 0 · tables: 3 · H2: 4 · Mermaid: 0 · TOC: yes

SCORE RATIONALE + GAP (V2)

579w (was 352) now with the full OTLP endpoint contract (protocol/auth/success/errors/limits/versioning) + redaction warning.

Future AGI

Docs

Integrations

Cookbooks

SDK

API

Search...

#K

Ask AI

Request demo

Dashboard >

Observe

Observe

Overview

GETTING STARTED

Quick start

CONCEPTS

Observability model

Traces

Spans

Sessions and users

OpenTelemetry

traceAI SDK

FEATURES

Trace explorer

Sessions

Users

Tags

Dashboards

Alerts and monitors

Trace evals

Voice observability

Manual tracing

REFERENCE

Filter syntax

Dashboard metrics

Export and endpoints

TROUBLESHOOTING

No traces appear

Spans and attributes

Dashboard numbers

Alerts not firing

Docs / Observe / Troubleshooting / No traces appear

Copy page

No traces appear

Your app ran but no trace shows up in FutureAGI Observe. The usual causes are an unflushed short-lived process, the wrong `project_type`, missing `FI_API_KEY` or `FI_SECRET_KEY`, or a date-picker window that is too narrow.

Symptom

You instrumented your app with `traceAI` and ran it, but no trace shows up in the `trace explorer`. Typically:

- A request ran with no error, but no new row appears in the trace list.
- A short script (a one-off `python app.py`) never produces a trace.
- Traces appeared before but stopped after a code change.

The most common cause is a short-lived process that exited before its spans flushed; the next most common are the wrong `project_type`, missing keys, or a date window that hides the trace. Work the checks below in order — the first one fixes the large majority of cases.

Quick checks

- The process **stays alive** long enough to export, or calls `force_flush()` before exiting.
- `FI_API_KEY` and `FI_SECRET_KEY` are set to this workspace's keys.
- `register()` is called with the correct `project_type` and `project_name`, **before** the framework client is created.
- The date picker is widened to **Today** (not the default 7-day window) and **Auto refresh** is on.

Causes and fixes

Cause	What you see	Fix
Short-lived process not flushed (most common)	A one-off script runs clean but no trace appears; long-running services are fine	Call <code>trace_provider.force_flush()</code> before the process ends, or pass <code>batch=False</code> to <code>register()</code> .
Wrong <code>project_type</code>	App runs, keys are valid, but traces never land in the project you expect	Set <code>project_type=ProjectType.OBSERVE</code> (and the matching <code>project_name</code>) in <code>register()</code> .

ON THIS PAGE

Symptom

Quick checks

Causes and fixes

Diagnostic commands

Minimal smoke test

Prevent recurrence

Next steps

Edit on GitHub

Send feedback

Ask a question

words: 606 w · code lines: 12 · screenshots: 1 · tables: 1 · H2: 7 · Mermaid: 0 · TOC: yes

SCORE RATIONALE + GAP (V2)

Symptom-first with causes/fixes table + 1 screenshot; add a verify-state shot or decision tree to reach 5.

Future AGI

Docs

Integrations

Cookbooks

SDK

API

Search...

Ask AI

Request demo

Dashboard

Observe

OVERVIEW

GETTING STARTED

CONCEPTS

FEATURES

REFERENCE

TROUBLESHOOTING

Overview

Quick start

Observability model

Traces

Spans

Sessions and users

OpenTelemetry

traceAI SDK

Trace explorer

Sessions

Users

Tags

Dashboards

Alerts and monitors

Trace evals

Voice observability

Manual tracing

Filter syntax

Dashboard metrics

Export and endpoints

No traces appear

Spans and attributes

Dashboard numbers

Alerts not firing

Docs / Observe / Troubleshooting / Spans and attributes

Copy page

Spans and attributes

The trace appears but spans are missing or fields like input/output are blank. Usual causes: masking is on, the instrumentor isn't attached, or a custom key isn't indexed.

Symptom

The trace shows up, but it's incomplete — a nested span is missing, or fields like input and output are blank, or a custom attribute you set isn't there. The usual causes are redaction being switched on (in which case blank is *expected*), the framework's instrumentor not being attached, or an attribute set on the wrong span or after it closed. Check redaction first, because a hidden field is working as designed, not a bug.

- A span's input/output show as hidden or blank.
- A framework's child spans (e.g. nested LangGraph nodes) don't appear.
- A custom attribute you set isn't on the span, or you can't filter by it.

Future AGI

Back

Save project

Auto refresh (30s)

Last updated on 5/19/2025, 08:02 p.m.

Trace

Sessions

Users

Get started

Primary Graph

Filter

Display

Add Eval

Dataset

Prompts

Agents

Prototypes

Knowledge base

Evals

Keys

Tracing

Error feed

Tests

Users

Annotations

Alerts

Dashboards

Gateway

Gateway

System

Resources

Docs

Help

No traces found

When spans or attributes aren't arriving, the trace list reads empty like this — the columns exist but nothing fills them. Confirm the instrumentor is attached and that redaction isn't hiding fields before assuming data was dropped.

words: 635 w · code lines: 8 · screenshots: 1 · tables: 1 · H2: 8 · Mermaid: 0 · TOC: yes

SCORE RATIONALE + GAP (V2)

635w, symptom-first with a causes/fixes table, and now a real error-state screenshot (empty / attribute-less trace view, 'No traces found'). Meets the troubleshooting bar (1-3 shots).

CARD 6 · TROUBLESHOOTING

TIER 2

Future AGI

Docs

Integrations

Cookbooks

SDK

API

Search...

Ask AI

Request demo

Dashboard >

Observe

OBSERVE

Overview

GETTING STARTED

Quick start

CONCEPTS

Observability model

Traces

Spans

Sessions and users

OpenTelemetry

traceAI SDK

FEATURES

Trace explorer

Sessions

Users

Tags

Dashboards

Alerts and monitors

Trace evals

Voice observability

Manual tracing

REFERENCE

Filter syntax

Dashboard metrics

Export and endpoints

TROUBLESHOOTING

No traces appear

Spans and attributes

Dashboard numbers

Alerts not firing

Docs / Observe / Troubleshooting / Dashboard numbers

Copy page

Dashboard numbers

A dashboard widget shows a number you didn't expect. Almost always it's the time range, granularity, aggregation, filters, or sampling — not bad data.

Symptom

A widget shows a number that doesn't match what you expected — cost too low, latency too high, a count that seems off. Almost always the data is right and the *query* is reading it differently than you assumed: the time range, granularity, aggregation, or filters change what a widget reports. Check those four before suspecting the underlying traces.

- A metric looks far higher or lower than reality.
- Two widgets that “should” match don't.
- A number changed when you only changed the time range or granularity.

Future AGI

Back

Agent Graph Demo

Auto refresh 150s

Default Workshop

Get started

Falcon AI

Dataset

Prompt

Agents

Prototype

Knowledge base

Evals

Keys

Observe

Tracing

Error Feed

Tasks

Chains

Annotations

Alerts

Dashboards

Gateway

System

Resources

Docs

Help

System Metrics

Latency

Tokens

Traffic

Cost

The System Metrics view. When a number here looks off, the data is usually right and the query is reading it differently — check the time range, granularity, aggregation, and filters before suspecting the traces.

Quick checks

ON THIS PAGE

- Symptom
- Quick checks
- Causes and fixes
- Diagnostic checks
- Minimal smoke test
- Escalate
- Prevent recurrence
- Next steps

- Edit on GitHub
- Send feedback
- Ask a question

words: 601 w · code lines: 0 · screenshots: 1 · tables: 1 · H2: 8 · Mermaid: 0 · TOC: yes

SCORE RATIONALE + GAP (V2)

601w, symptom-first + causes/fixes table + a real System Metrics dashboard screenshot (frontend-mocked, synthetic data). Meets the bar.

Future AGI

Docs

Integrations

Cookbooks

SDK

API

Search...

Ask AI

Request demo

Dashboard

Observe

OVERVIEW

GETTING STARTED

CONCEPTS

FEATURES

REFERENCE

TROUBLESHOOTING

Alerts not firing

Docs / Observe / Troubleshooting / Alerts not firing

Copy page

Alerts not firing

A metric crossed your threshold but no alert arrived. Usual causes: the monitor's frequency hasn't elapsed, it's muted, the threshold direction is wrong, or notifications are misconfigured.

Symptom

A metric crossed what you thought was the threshold, but no email or Slack arrived. The usual causes are timing (the monitor only evaluates on its schedule), the monitor being muted, the threshold direction or value being set differently than you remember, or the notification channel itself failing. Check the schedule and mute state first — those explain most “missing” alerts.

- A metric clearly breached the limit but no notification came.
- Alerts used to arrive and stopped.
- The alert log shows nothing for the period you expected.

Future AGI

Back

Agent Graph Demo

LLM Tracing

Evals & Tasks

Charts

Alerts

Get started

Search

View Docs

New Alert

Build	Status	Alert Type	Last Triggered	No. of Triggers	Updated at
Issues	Active	Latency	-	0	4 days ago
High latency (p95 > 2s)					

Dataset

Prompts

Agents

Prototype

Knowledge base

Evals

Keys

Observes

Tracing

Error Feed

Tasks

Users

Annotations

Alerts

Dashboards

Gateway

Gateway

Systems

Resources

Docs

Help

Rows per page: 25 1-1 of 1

An alert configured but never fired: Status is Active, yet Last Triggered shows '-' and the trigger count is 0. That points to schedule timing, a mute, or a threshold set differently than expected — not a broken metric.

ON THIS PAGE

- Symptom
- Quick checks
- Causes and fixes
- Diagnostic checks
- Minimal smoke test
- Escalate
- Prevent recurrence
- Next steps

- Edit on GitHub
- Send feedback
- Ask a question

words: 564 w · code lines: 0 · screenshots: 1 · tables: 1 · H2: 8 · Mermaid: 0 · TOC: yes

SCORE RATIONALE + GAP (V2)

564w, symptom-first + causes/fixes table + a real screenshot of an Active alert that never fired (Last Triggered '-', 0 triggers). Meets the bar.

traceAI — SDK section

11 pages · v3 average 4.18/5 (v3 4.18, ▲ +0.00)

Future AGI

Docs

Integrations

Cookbooks

SDK

API

Q Search... #K

Ask AI

Request demo

Dashboard >

</> traceAI

TRACEAI

Overview

Quick start

CONCEPTS

OpenTelemetry

traceAI SDK

MANUAL INSTRUMENTATION

Set up tracing

Instrument with traceAI helpers

Get the current span

Add attributes and metadata

Log prompt templates

Events and exceptions

Set session and user

Create tool spans

Mask span attributes

Advanced tracing

Inline evals

Annotate spans

Import from Langfuse

FRAMEWORK INTEGRATIONS

Overview

> LLM providers

> Frameworks and agents

> Voice and realtime

> Java

> Other

REFERENCE

Semantic conventions

TROUBLESHOOTING

Spans not exported

Docs / traceAI: the instrumentation SDK for FutureAGI

Copy page

traceAI: the instrumentation SDK for FutureAGI

traceAI is FutureAGI's open instrumentation SDK. It captures your AI app as OpenTelemetry traces — automatically for 30+ frameworks, or manually for full control — and exports them to Observe.

About

traceAI is FutureAGI's instrumentation SDK — the layer that turns your running AI application into traces. It builds on OpenTelemetry, so the spans it emits are a standard format, not a proprietary one. You instrument once, either automatically for a supported framework or manually for full control, and every request your app handles is exported as a trace that Observe reads. traceAI is the SDK that produces the data; Observe is the product that lets you search, replay, score, and alert on it.

Why traceAI

Three reasons it's worth instrumenting with traceAI rather than rolling your own:

- It's **standard OpenTelemetry, not a proprietary format**. The spans go to FutureAGI today, but the same instrumentation works with any OTel-compatible backend — so you're not locked in, and a trace looks the same wherever it's read.
- Auto-instrumentation is one line per framework**. Add the instrumentor for OpenAI, LangChain, LlamaIndex, and others, and your model, tool, and retrieval calls become traces with no span code in your application.
- Manual instrumentation is there when you need control**. Where an instrumentor doesn't reach — your own functions, custom retrieval, glue logic — you create spans by hand, and they nest into the same traces. You get coverage without giving up precision.

What you can do here

- Auto-instrument a framework** — add one instrumentor and capture LangChain, LlamaIndex, CrewAI, OpenAI, Anthropic, and 30+ others with no manual span code.
- Instrument manually** — create your own spans, attributes, and events when auto-instrumentation doesn't reach your code.
- Set session, user, and tag attributes** — enrich spans so Observe can group and filter them.

ON THIS PAGE

About

Why traceAI

What you can do here

Get started

Featured concepts

Featured how-tos

Where next

Edit on GitHub

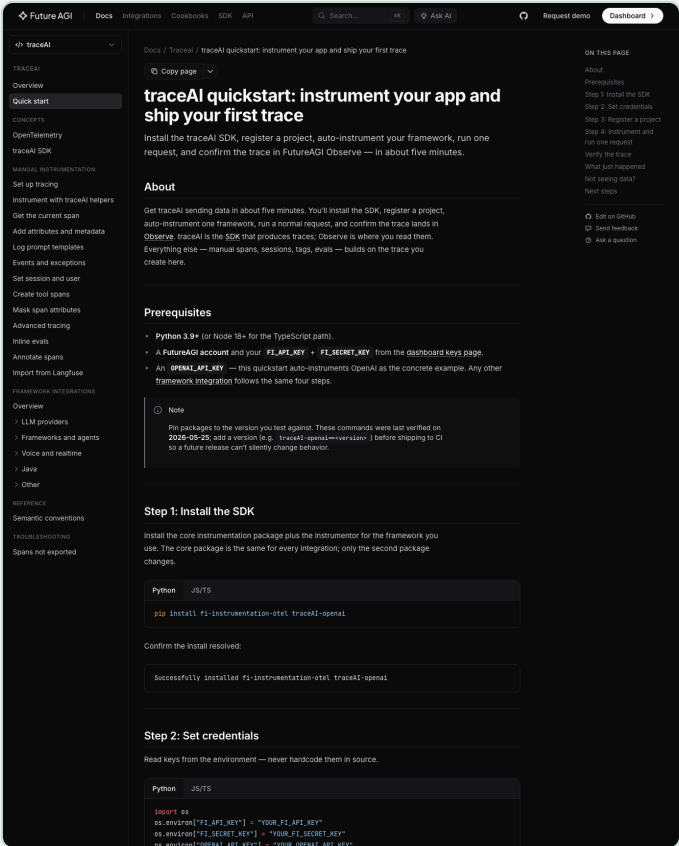
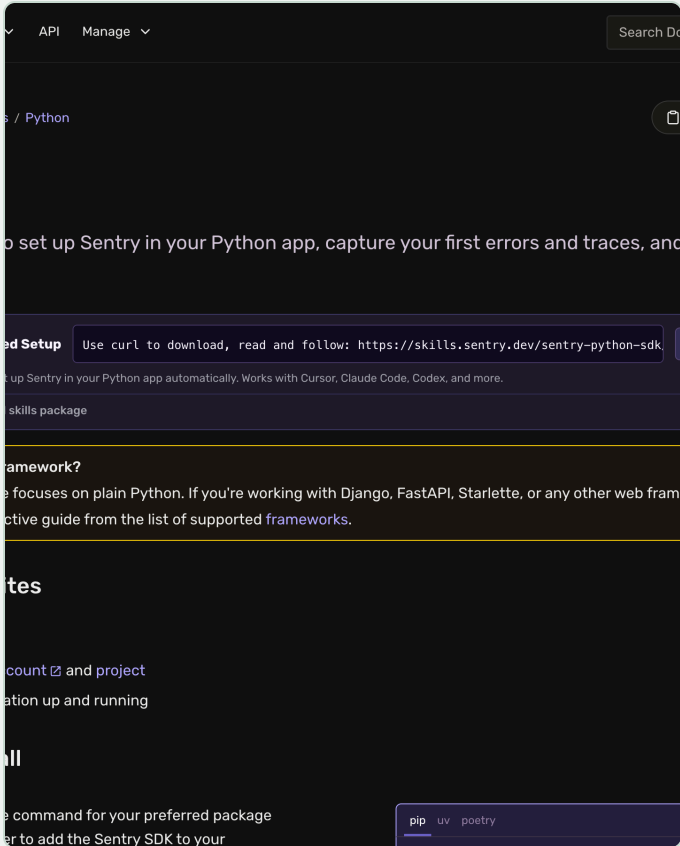
Send feedback

Ask a question

words: 479 w (floor 400–900) · code lines: 0 · screenshots: 0 · tables: 0 · H2: 7 · Mermaid: 0 · TOC: yes

SCORE RATIONALE + GAP (V2)

479w (clears 400 floor) with a 'why traceAI' section, what-you-can-do, get-started card, and concept/how-to card grids.



words: 854 w (floor 800–1200) · code lines: 52 · screenshots: 1 · tables: 1 · H2: 10 · Mermaid: 0 · TOC:

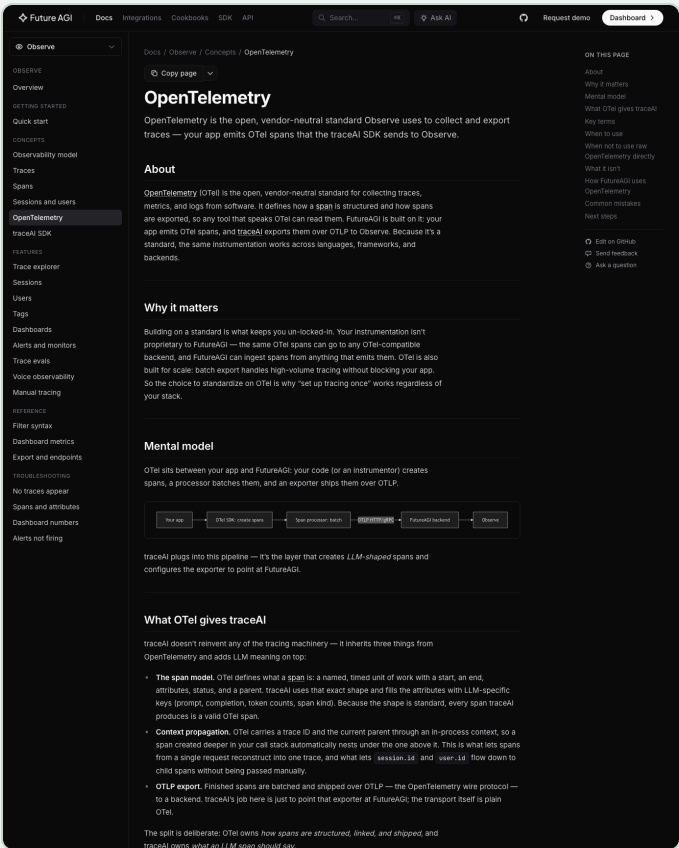
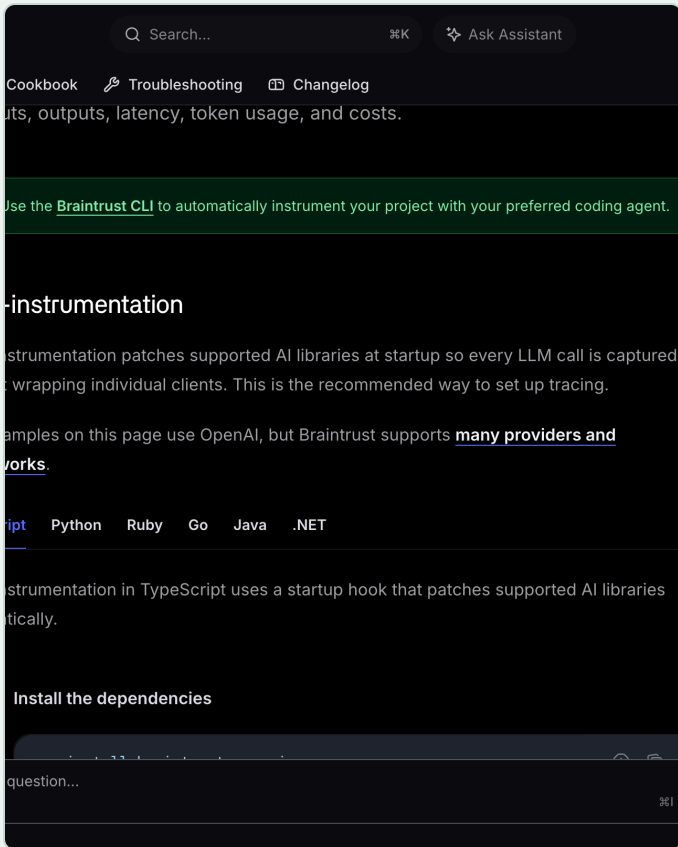
yes

SCORE RATIONALE + GAP (V2)

WAS A 404 — now a full quickstart: 854w, 4 numbered steps with expected-output each, Verify, 'what just happened', Next steps; single reused screenshot keeps it off 5.

BENCHMARK EXEMPLAR · PAIR 4 — BRAINTRUST DENSITY
· 5/5

OUR PAGE (V4) — TRACEAI CONCEPTS



words: **986** w (floor 800–1500) · code lines: **15** · screenshots: **0** · tables: **1** · H2: **11** · Mermaid: **1** · TOC:

yes

SCORE RATIONALE + GAP (V2)

Shared Observe OpenTelemetry concept, now 986w with when-not; traceAI reuses the Observe concept pages (no distinct page) so it's a 4.

Manual instrumentation (14 pages — rep: Set up tracing)

#28

/docs/traceai/manual-instrumentation/set-up-tracing

4 → 4

4/5

CARD 4 · FEATURE DEEP-DIVE

TIER 2

Future AGI

Docs

Integrations

Cookbooks

SDK

API

Search...

Ask AI

Request demo

Dashboard

Observe

OBSERVE

Overview

GETTING STARTED

Quick start

CONCEPTS

Observability model

Traces

Spans

Sessions and users

OpenTelemetry

traceAI SDK

FEATURES

Trace explorer

Sessions

Users

Tags

Dashboards

Alerts and monitors

Trace evals

Voice observability

Manual tracing

REFERENCE

Filter syntax

Dashboard metrics

Export and endpoints

TROUBLESHOOTING

No traces appear

Spans and attributes

Dashboard numbers

Alerts not firing

Docs / Traceai / Manual Instrumentation / Set Up Tracing with OpenTelemetry

Copy page

Set Up Tracing with OpenTelemetry

Connect your app to FutureAGI: register a tracer provider, add an auto-instrumentor or manual OpenTelemetry spans, and confirm traces arrive in your project.

About

Tracing captures every LLM call, tool invocation, or custom operation in your application and sends it to Future AGI. You set it up by calling `register()` once to connect your app to a project. Then you add instrumentation, either auto-instrumentors for supported frameworks (OpenAI, LangChain, etc.) or manual spans for custom logic. Once connected, all captured data appears in your project dashboard where you can inspect traces, run evals, and set up alerts.

When to use

- Production monitoring:** Register an Observe project and auto-instrument LLM calls so every request is traced with latency, cost, and token usage.
- Experiment tracking:** Register an Experiment project with eval tags and version names to compare prompt or model changes across runs.
- Custom spans:** Use `FITracer` to manually create spans for operations that auto-instrumentors don't cover.
- Privacy control:** Use `TraceConfig` to redact sensitive inputs, outputs, or messages before they leave your app.
- Any Python or JS/TS app:** Works with any application via OpenTelemetry. Auto-instrumentors cover a wide range of frameworks — see the [Auto Instrumentation catalog](#).

Prerequisites

- Python 3.9+ or Node 18+.**
- A FutureAGI account with your `FI_API_KEY` and `FI_SECRET_KEY` from the [dashboard](#).
- The core `fi-instrumentation-otel` package plus the instrumentor for your framework (e.g. `traceAI-openai`).

ON THIS PAGE

About

When to use

Prerequisites

How to

Verify

Troubleshooting

Key concepts

Next Steps

Edit on GitHub

Send feedback

Ask a question

words: 1003 w (floor 900–1800) · code lines: 67 · screenshots: 0 · tables: 5 · H2: 8 · Mermaid: 0 · TOC:

yes

SCORE RATIONALE + GAP (V2)

Representative (set-up-tracing): code trimmed 118→67 (under the 80 cap), 1003w, 5 tables; 0 screenshots (feature wants 2–4) keeps it at 4.

Future AGI

Docs

Integrations

Cookbooks

SDK

API

Search...

#K

Ask AI

Request demo

Dashboard >

</> traceAI

TRACEAI

Overview

Quick start

CONCEPTS

OpenTelemetry

traceAI SDK

MANUAL INSTRUMENTATION

Set up tracing

Instrument with traceAI helpers

Get the current span

Add attributes and metadata

Log prompt templates

Events and exceptions

Set session and user

Create tool spans

Mask span attributes

Advanced tracing

Inline evals

Annotate spans

Import from Langfuse

FRAMEWORK INTEGRATIONS

Overview

> LLM providers

> Frameworks and agents

> Voice and realtime

> Java

> Other

REFERENCE

Semantic conventions

TROUBLESHOOTING

Spans not exported

Docs / Traceai / Auto-Instrumentation Integrations: Future AGI Tracing

Copy page

Auto-Instrumentation Integrations: Future AGI Tracing

Auto-instrumentation integrations for LLM frameworks in Python, JavaScript, and Java. Install a traceAI package to start capturing spans automatically.

About

Auto-Instrumentation adds tracing to your AI application with minimal code changes. Install the `traceAI` package for your framework, [register](#) a trace provider, and FutureAGI captures spans, inputs, outputs, latency, token counts, and cost automatically — no per-call span code. The result is the same [trace](#) you'd build by hand, without editing your business logic.

Reach for auto-instrumentation whenever you use a supported framework — it's the fastest path and the right default. Drop to [manual instrumentation](#) only for code a framework wrapper can't see: your own functions, custom tools, or steps between framework calls. The two compose — auto-instrument the framework, then add manual spans where you need them.

How it works

Python and JS/TS integrations use **instrumentors** that patch the client library at startup, so every call it makes is traced. Java integrations use explicit **Traced*** **wrappers** around your existing clients. Both emit the same OpenTelemetry spans, so a [LangChain](#) run and a raw [OpenAI](#) call land in [Observe](#) with the same shape. The setup is always the same four steps — install the package, set credentials, register a provider with `project_type=OBSERVE`, attach the instrumentor — covered end to end in the [quick start](#). After a run, confirm the trace in [Observe → Tracing](#); if nothing arrives, see [Spans not exported](#).

Pick your framework below. Each page covers install, configuration, the spans it produces, and how to verify them.

LLM Providers

[OpenAI](#)

[Anthropic](#)

[AWS Bedrock](#)

ON THIS PAGE

About

How it works

LLM Providers

Frameworks & Agents

Voice & Realtime

Java

Other

Edit on GitHub

Send feedback

Ask a question

words: 469 w (floor 400–900) · code lines: 0 · screenshots: 0 · tables: 0 · H2: 7 · Mermaid: 0 · TOC: yes

SCORE RATIONALE + GAP (V2)

469w (clears 400 floor) with how-auto-works + auto-vs-manual + a verify pointer; catalog card grids present.

Future AGI

Docs

Integrations

Cookbooks

SDK

API

Q Search...

Ask AI

Request demo

Dashboard

traceAI

TRACEAI

Overview

Quick start

CONCEPTS

OpenTelemetry

traceAI SDK

MANUAL INSTRUMENTATION

Set up tracing

Instrument with traceAI helpers

Get the current span

Add attributes and metadata

Log prompt templates

Events and exceptions

Set session and user

Create tool spans

Mask span attributes

Advanced tracing

Inline evals

Annotate spans

Import from Langfuse

FRAMEWORK INTEGRATIONS

Overview

LLM providers

OpenAI

Anthropic

AWS Bedrock

Vertex AI

Google GenAI

Google ADK

Groq

MistralAI

Together AI

Ollama

Portkey

Frameworks and agents

Docs / Traceai / Auto / OpenAI Tracing with Future AGI: Auto-Instrumentation

Copy page

OpenAI Tracing with Future AGI: Auto-Instrumentation

Set up auto-instrumentation for OpenAI with Future AGI tracing. Install traceAI-openai to capture chat completion, embedding, and tool call spans.

1. Installation

First install the traceAI package to access the observability framework

Python JS/TS

```
pip install traceAI-openai
```

2. Set Environment Variables

Set up your environment variables to authenticate with both FutureAGI and OpenAI services.

Python JS/TS

```
import os

os.environ["OPENAI_API_KEY"] = "your-openai-api-key"
os.environ["FI_API_KEY"] = "your-futureagi-api-key"
os.environ["FI_SECRET_KEY"] = "your-futureagi-secret-key"
```

3. Initialize Trace Provider

Set up the trace provider to create a new project in FutureAGI, establish telemetry data pipelines .

Python JS/TS

```
from fi_instrumentation import register
from fi_instrumentation.fi_types import ProjectType
```

ON THIS PAGE

1. Installation

2. Set Environment Variables

3. Initialize Trace Provider

4. Instrument your Project

5. Interact with OpenAI

Chat Completion

Audio and speech

Image Generation

Chat Streaming

What spans are produced

Verify the trace

Troubleshooting

Related

Edit on GitHub

Send feedback

Ask a question

words: 535w < 900 floor · code lines: 136 · screenshots: 0 · tables: 1 · H2: 9 · Mermaid: 0 · TOC: yes

SCORE RATIONALE + GAP (V2)

Now has What-spans-are-produced + Verify + Troubleshooting table + Span-concept link (3→4). Prose ~535w is under the 900 integration floor and inline code (136 lines) still exceeds 80 — accepted: \$5 structure applied, floor/code-cap not closed without padding/cutting examples.

Future AGI

Docs

Integrations

Cookbooks

SDK

API

Q Search...

Ask AI

Request demo

Dashboard >

</> traceAI

TRACEAI

Overview

Quick start

CONCEPTS

OpenTelemetry

traceAI SDK

MANUAL INSTRUMENTATION

Set up tracing

Instrument with traceAI helpers

Get the current span

Add attributes and metadata

Log prompt templates

Events and exceptions

Set session and user

Create tool spans

Mask span attributes

Advanced tracing

Inline evals

Annotate spans

Import from Langfuse

FRAMEWORK INTEGRATIONS

Overview

> LLM providers

< Frameworks and agents

LangChain

LangGraph

LlamaIndex

LlamaIndex Workflows

LiteLLM

CrewAI

AutoGen

Haystack

DSPy

OpenAI Agents

Smol Agents

Docs / Traceai / Auto / LangChain Tracing with Future AGI: Auto-Instrumentation

Copy page

LangChain Tracing with Future AGI: Auto-Instrumentation

Set up auto-instrumentation for LangChain with Future AGI tracing. Install traceAI-langchain to capture chain, tool, and LLM call spans.

1. Installation

First install the traceAI package and necessary LangChain packages.

Python

JS/TS

```
pip install traceAI-langchain
pip install langchain_openai
```

2. Set Environment Variables

Set up your environment variables to authenticate with both FutureAGI and OpenAI.

Python

JS/TS

```
import os

os.environ["OPENAI_API_KEY"] = "your-openai-api-key"
os.environ["FI_API_KEY"] = "your-futureagi-api-key"
os.environ["FI_SECRET_KEY"] = "your-futureagi-secret-key"
```

3. Initialize Trace Provider

Set up the trace provider to create a new project in FutureAGI, establish telemetry data pipelines .

Python

JS/TS

```
from fi_instrumentation import register
from fi_instrumentation.fi_types import ProjectType
```

ON THIS PAGE

1. Installation

2. Set Environment Variables

3. Initialize Trace Provider

4. Instrument your Project

5. Create LangChain Components

What spans are produced

Verify the trace

Troubleshooting

Related

Edit on GitHub

Send feedback

Ask a question

Future AGI

Docs

Integrations

Cookbooks

SDK

API

Search...

#K

Ask AI

Request demo

Dashboard >

</> traceAI

TRACEAI

Overview

Quick start

CONCEPTS

OpenTelemetry

traceAI SDK

MANUAL INSTRUMENTATION

Set up tracing

Instrument with traceAI helpers

Get the current span

Add attributes and metadata

Log prompt templates

Events and exceptions

Set session and user

Create tool spans

Mask span attributes

Advanced tracing

Inline evals

Annotate spans

Import from Langfuse

FRAMEWORK INTEGRATIONS

Overview

> LLM providers

> Frameworks and agents

> Voice and realtime

LiveKit

Pipecat

> Java

> Other

REFERENCE

Semantic conventions

TROUBLESHOOTING

Spans not exported

Docs / Traceai / Auto / LiveKit Tracing with Future AGI: Voice Agent Observability

Copy page

LiveKit Tracing with Future AGI: Voice Agent Observability

Integrate LiveKit with Future AGI for voice agent observability. Trace real-time voice interactions and monitor agent performance with traceAI-livekit.

1. Installation

Install the traceAI and LiveKit agent packages to enable voice agent capabilities with observability.

```
pip install traceai-livekit
pip install livekit
pip install python-dotenv
```

2. Set Environment Variables

Set up your environment variables to authenticate with both FutureAGI and LiveKit services.

```
# .env file
FI_API_KEY=your-futureagi-api-key
FI_SECRET_KEY=your-futureagi-secret-key
OPENAI_API_KEY=your-openai-api-key
LIVEKIT_API_KEY=your-livekit-api-key
LIVEKIT_API_SECRET=your-livekit-api-secret
```

3. Create Your Agent

Create a voice assistant agent by extending the LiveKit Agent class with your custom instructions.

```
import logging

from dotenv import load_dotenv
from livekit.agents import import (
```

ON THIS PAGE

1. Installation

2. Set Environment Variables

3. Create Your Agent

4. Initialize Trace Provider

5. Implement the Agent Session

6. Run Your Agent

Complete Example

What spans are produced

Verify the trace

Troubleshooting

Related

Edit on GitHub

Send feedback

Ask a question

words: 496w < 900 floor · code lines: 181 · screenshots: 0 · tables: 1 · H2: 11 · Mermaid: 0 · TOC:

yes

SCORE RATIONALE + GAP (V2)

Same §5 sections added. ~496w prose under floor; inline code 181 lines exceeds 80 (voice example) — accepted overflow.

Future AGI

Docs

Integrations

Cookbooks

SDK

API

Q Search...

#K

Ask AI

Request demo

Dashboard >

</> traceAI

TRACEAI

Overview

Quick start

CONCEPTS

OpenTelemetry

traceAI SDK

MANUAL INSTRUMENTATION

Set up tracing

Instrument with traceAI helpers

Get the current span

Add attributes and metadata

Log prompt templates

Events and exceptions

Set session and user

Create tool spans

Mask span attributes

Advanced tracing

Inline evals

Annotate spans

Import from Langfuse

FRAMEWORK INTEGRATIONS

Overview

> LLM providers

> Frameworks and agents

> Voice and realtime

< Java

Overview

Spring Boot

OpenAI

Anthropic

AWS Bedrock

Cohere

Pinecone

LLM Providers

Vector Databases

Docs / Traceai / Auto / Java SDK: TraceAI Setup and Instrumentation with Future AGI

Copy page

Java SDK: TraceAI Setup and Instrumentation with Future AGI

Set up TraceAI for Java applications. Initialize the tracer, configure credentials, and instrument your LLM clients, vector databases, and frameworks.

TL;DR

- TraceAI.init() or TraceAI.initFromEnvironment() to start
- Every integration is a Traced<X> wrapper around your existing client
- Spans export to FutureAGI via OTLP HTTP, batched every 5 seconds
- Thread-local context (session, user, tags) applied to all spans in scope
- Distributed via JitPack (Maven/Gradle)

How it works

The Java SDK wraps your existing clients with Traced<X> classes. You initialize TraceAI once, then wrap each client you want to trace. The wrappers delegate every call to the original client and create OpenTelemetry spans around it - capturing inputs, outputs, token counts, latency, and errors.

```
// 1. Initialize once
TraceAI.init(TraceConfig.builder()
    .baseUrl("https://api.futureagi.com")
    .apiKey(System.getenv("FI_API_KEY"))
    .secretKey(System.getenv("FI_SECRET_KEY"))
    .projectName("my-project")
    .build());

// 2. Wrap your client
OpenAIClient client = OpenAIOkHttpClient.builder()
    .apiKey(System.getenv("OPENAI_API_KEY"))
    .build();
TracedOpenAIClient traced = new TracedOpenAIClient(client);

// 3. Use it normally - spans are created automatically
ChatCompletion response = traced.createChatCompletion(params);
```

Installation

ON THIS PAGE

How it works

Installation

Initialization

From code

From environment variables

Getting the tracer

TraceConfig reference

FITracer methods

Available methods

FISpanKind

Context attributes

Shutdown

Available integrations

What spans are produced

Verify the trace

Troubleshooting

Related

Edit on GitHub

Send feedback

Ask a question

words: 1145 w (floor 900–1500) · code lines: 95 · screenshots: 0 · tables: 4 · H2: 13 · Mermaid: 0 · TOC: yes

SCORE RATIONALE + GAP (V2)
Rewritten to the leaf pattern: now 1145w with 'What spans are produced' (FISpanKind), a Verify step, a Troubleshooting table, and a Span-concept link. Inline code 95 lines still over the 80 cap — accepted for a comprehensive Java setup reference.

Future AGI

Docs

Integrations

Cookbooks

SDK

API

Search...

Ask AI

Request demo

Dashboard

traceAI

TRACEAI

Overview

Quick start

CONCEPTS

OpenTelemetry

traceAI SDK

MANUAL INSTRUMENTATION

Set up tracing

Instrument with traceAI helpers

Get the current span

Add attributes and metadata

Log prompt templates

Events and exceptions

Set session and user

Create tool spans

Mask span attributes

Advanced tracing

Inline evals

Annotate spans

Import from Langfuse

FRAMEWORK INTEGRATIONS

Overview

LLM providers

Frameworks and agents

Voice and realtime

Java

Other

REFERENCE

Semantic conventions

TROUBLESHOOTING

Spans not exported

Docs / Traceai / Manual Instrumentation / Semantic Conventions: Standard Span Attribute Keys

Copy page

Semantic Conventions: Standard Span Attribute Keys

The standardized attribute keys for spans — span kinds, message, document, embedding, and tool-call attributes — that keep trace data consistent and queryable.

About

Every LLM provider returns data in a different format. Without a standard set of attribute keys, the same concept (model name, token count, input messages) ends up stored differently depending on which provider or framework was used, making filtering and comparison impossible. FI Semantic Conventions define a single set of attribute keys that the Future AGI platform recognizes. When spans carry these keys, they are highlighted in the UI and enable filtering, search, and analytics across providers.

When to use

- Consistent tracing:** Standardized keys across different LLM providers and frameworks so trace data is uniform and comparable.
- LLM data capture:** Record model name, token counts, input/output messages, and prompt templates in a structured, queryable schema.
- Filtering and search:** Filter and search traces in the Future AGI dashboard using well-known attribute keys.
- Retrieval and reranker tracing:** Attach document scores, query strings, and model names to retrieval and reranker spans for RAG pipeline visibility.
- Session and user analytics:** Use `session.id` and `user.id` to group traces and run per-user analytics.

How to

1

Install the package

Install the traceAI instrumentation package to access semantic convention constants.

Python

Javascript

```
pip install fi-instrumentation-otel
```

ON THIS PAGE

About

When to use

How to

Attribute overview

Key concepts

Next Steps

Edit on GitHub

Send feedback

Ask a question

words: 1253 w · code lines: 402 · screenshots: 0 · tables: 1 · H2: 6 · Mermaid: 0 · TOC: yes

SCORE RATIONALE + GAP (V2)

1253w, 20 code blocks / 402 lines as the artifact, attribute tables, 0 screenshots (correct) — a real SDK/semantic-conventions reference.

Future AGI

Docs

Integrations

Cookbooks

SDK

API

Search...

Ask AI

Request demo

Dashboard

traceAI

TRACEAI

Overview

Quick start

CONCEPTS

OpenTelemetry

traceAI SDK

MANUAL INSTRUMENTATION

Set up tracing

Instrument with traceAI helpers

Get the current span

Add attributes and metadata

Log prompt templates

Events and exceptions

Set session and user

Create tool spans

Mask span attributes

Advanced tracing

Inline evals

Annotate spans

Import from Langfuse

FRAMEWORK INTEGRATIONS

Overview

LLM providers

Frameworks and agents

Voice and realtime

Java

Other

REFERENCE

Semantic conventions

TROUBLESHOOTING

Spans not exported

Docs / Traceai / Troubleshooting / Spans not exported

Copy page

Spans not exported

Your app runs but no spans reach FutureAGI. Work through the common causes — unflushed batches, wrong project type, missing credentials, and instrumentor order — with the exact fix for each.

Symptom

Your application runs without errors, but no spans appear in **Observe** — the trace list stays empty, or new requests never show up. The instrumentation looks wired in, yet nothing is exported.

Quick checks

- The process **stays alive** long enough to flush, or you call `force_flush()` before it exits.
- `register(...)` ran with `project_type=ProjectType.OBSERVE` **before** any model client was created.
- `FI_API_KEY` and `FI_SECRET_KEY` are set in the environment the app actually runs in.
- The instrumentor (`.instrument(...)`) was attached to the **same** tracer provider `register` returned.

Causes and fixes

Cause	What you see	Fix
Batch exporter never flushed	A short script or serverless function exits immediately; spans are still buffered.	Call <code>trace_provider.force_flush()</code> (and <code>shutdown()</code>) before the process ends. The batch exporter sends on an interval, so a process that exits in under a second loses its buffer.
Wrong project type	Traces go somewhere, but not to your Observe project.	Pass <code>project_type=ProjectType.OBSERVE</code> to <code>register</code> . A project registered as another type won't appear in Observe.
Missing or wrong credentials	Export silently fails or 401s in logs.	Set <code>FI_API_KEY</code> / <code>FI_SECRET_KEY</code> for this workspace; confirm they match the keys page .
Instrumented	The model client was created before	Call <code>register</code> and <code>instrument()</code> at startup, before

ON THIS PAGE

Symptom

Quick checks

Causes and fixes

Diagnostic commands

Minimal smoke test

Escalate

Prevent recurrence

Next steps

Edit on GitHub

Send feedback

Ask a question

words: 501 w · code lines: 3 · screenshots: 0 · tables: 1 · H2: 8 · Mermaid: 0 · TOC: yes

SCORE RATIONALE + GAP (V2)

Strong symptom-first structure (Symptom → Quick checks → Causes/fixes → Diagnostics → Smoke test → Escalate → Prevent); 0 screenshots — blocked live-dashboard state (maintainer note in source).